CSE3001
Database Management Systems

Modules	Syllabus
Module-4	**PL/SQL:** Declaring PL/SQL Variables, Writing Executable Statements, Using SQL Statements Within a PL/SQL Block, Control Structures, Composite Data Types, Using Explicit Cursors, Handling Exceptions, Introducing Stored Procedures and Functions. **Data Storage:** Overview of Physical Storage Media - Magnetic disk Flash storage - RAID-File and Record Organization-Indexing and Hashing, Ordered Indices - B+Tree Index File-Static Hashing -Dynamic Hashing Query Processing: Overview-measures of Query Cost. **Query Optimization Techniques**- Cost based Optimization-Heuristic Optimization

PL/SQL — Overview

- The PL/SQL programming language was developed by Oracle Corporation in the late 1980s as Oracle relational database.
- Stands for Procedural Language extension to SQL.



Following are certain notable facts about PL/SQL:

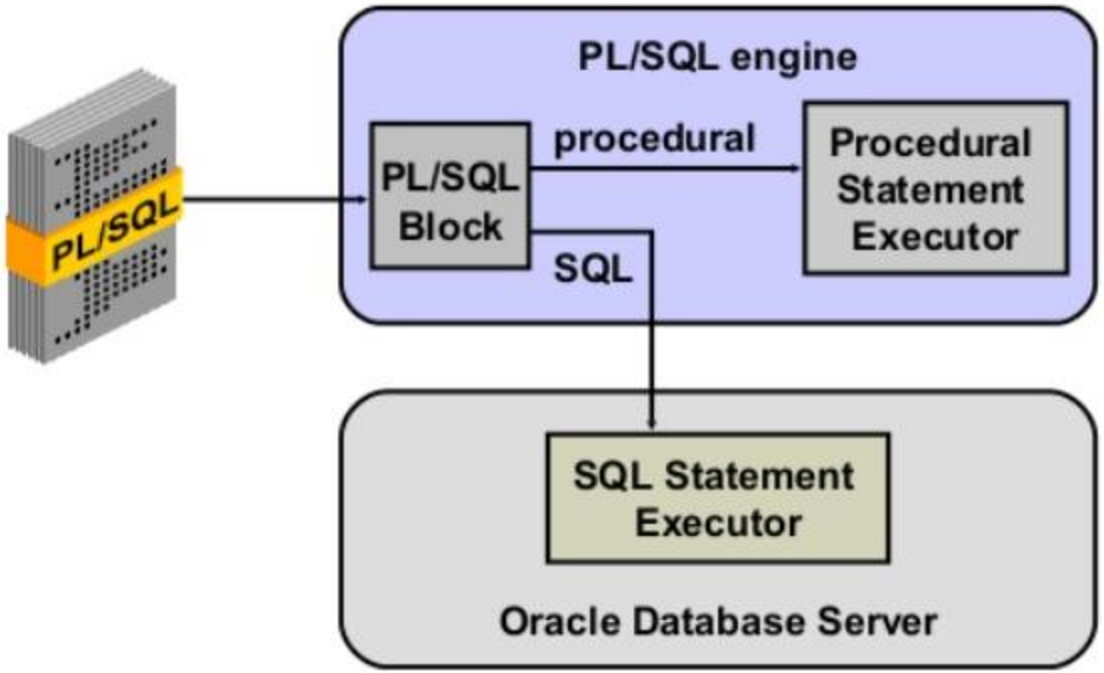
- PL/SQL is a completely portable, high-performance transaction-processing language.
- PL/SQL provides a built-in, interpreted and OS independent programming environment.
- PL/SQL can also directly be called from the command-line SQL*Plus interface.

Features of PL/SQL

PL/SQL has the following features:

- PL/SQL is tightly integrated with SQL.
- It offers extensive error checking.
- It offers numerous data types.
- It supports structured programming through functions and procedures.
- It supports object-oriented programming. So it saves time on design and debugging by strong features, such as exception handling, encapsulation, data hiding, and object-oriented data types.
- It supports the development of web applications and server pages.

PL/SQL — Environment Setup



PL/SQL — Basic Syntax

Following is the basic structure of a PL/SQL block:

DECLARE <declarations section>	Starts with the keyword **DECLARE**. It is an optional section & defines all variables, cursors, subprograms and other elements to be used in the program.
BEGIN <executable command(s)>	This section is enclosed between the keywords **BEGIN** and **END** and it is a mandatory section. It consists of the executable PL/SQL statements of the program.
EXCEPTION <exception handling>	This section starts with the keyword **EXCEPTION**. This optional section contains **exception(s)** that handle errors in the program.
END;	

The 'Hello World' Example of PL/SQL

```
DECLARE
   message varchar2(20):= 'Hello, World!';
BEGIN
   dbms_output.put_line (message);
END;
/
```

Output−

```
Hello World

PL/SQL procedure successfully completed.
```

PL/SQL - Variables

- PL/SQL variables must be declared in the declaration section.

- The syntax for declaring a variable is-

  variable_name datatype [NOT NULL] [:= | DEFAULT initial_value]

```
DECLARE
   a integer := 10;
   b integer := 20;
   c integer;

BEGIN
   c := a + b;
   dbms_output.put_line('Value of c: ' || c);

END;
/
```

PL/SQL – Conditional Statements

Write a program to find largest number from given numbers-

```
declare
a number := &a;
b number := &b;
c number := &c;

begin
if a>b and a>c
then
dbms_output.put_line('greatest number is ' ||a);

elsif b>a and b>c
then
dbms_output.put_line('greatest number is ' ||b);

else
dbms_output.put_line('greatest number is ' ||c);
end if;
end;
/
```

PL/SQL – Iterative Statements

Write program to generate 10 numbers using-
1. Simple Loop
2. While Loop
3. For Loop

1. Simple Loop

```
declare
i number;
begin
i:=1;
loop
dbms_output.put_line(i);
i:=i+1;
exit when i>10;
end loop;
end;
/
```

2. While Loop

```
declare
i number;
begin
i:=1;
while(i<10)
loop
dbms_output.put_line(i);
i:=i+1;
end loop;
end;
/
```

3. For Loop

```
declare
i number;
begin
i:=1;
for i in 1..10
loop
dbms_output.put_line(i);
end loop;
end;
/
```

Assigning SQL Query Results to PL/SQL Variables

Lets consider a relation- **customers(id, name, address, salary)**

The following program assigns values from the above table to PL/SQL variables using the SELECT INTO clause of SQL −

```
DECLARE
   c_id customers.id%type := 1;
   c_name  customers.name%type;
   c_addr customers.address%type;
   c_sal  customers.salary%type;

BEGIN
   SELECT name, address, salary INTO c_name, c_addr, c_sal  FROM customers
   WHERE id = c_id;
   dbms_output.put_line ('Customer ' ||c_name || ' from ' || c_addr || ' earns ' || c_sal);

END;
/
```

Output-

Customer Ramesh from Ahmedabad earns 2000
PL/SQL procedure completed successfully

PL/SQL – %type & %rowtype

```
declare
veid emp.eid%type:=&veid;
vename emp.ename%type;
vdno emp.dno%type;
begin
select eid,ename,dno into veid,vename,vdno from emp where eid=veid;
dbms_output.put_line(veid||vename||vdno);
end;
/
```

```
declare
my_emprow emp%rowtype;
no number:=&no;
begin
select * into my_emprow from emp where eid=no;
dbms_output.put_line(my_emprow.eid||my_emprow.ename||my_emprow.dno);
end;
/
```

PL/SQL – Records

- Records are another type of data structure.
- Records are composite data type which is combination of char, varchar, number etc.
- It can be visualize as row of table.

- For example, you want to keep track of your books in a library. You might want to track the following attributes about each book, such as Title, Author, Subject, Book ID. A record containing a field for each of these items allows treating a BOOK as a logical unit and allows you to organize and represent its information in a better way.

PL/SQL can handle the following types of records −

- Table-based
- Cursor-based records
- User-defined records

PL/SQL – Table Based Records

- The %ROWTYPE attribute enables a programmer to create table-based and cursor based records.

The following example illustrates the concept of table-based records.

```
DECLARE
   emp_rec emp%rowtype;
BEGIN
   SELECT * into emp_rec
   FROM emp
   WHERE eid = 5;
   dbms_output.put_line('Emp ID: ' || emp_rec.eid);
   dbms_output.put_line('Emp Name: ' || emp_rec.ename);
   dbms_output.put_line('Emp Dept: ' || emp_rec.dno);
END;
/
```

PL/SQL – Cursor Based Records

The following example illustrates the concept of cursor-based records.

```
DECLARE
  CURSOR emp_cur is
    SELECT eid, ename, dno FROM emp;
  emp_rec emp_cur%rowtype;
BEGIN
  OPEN emp_cur;
  LOOP
    FETCH emp_cur into emp_rec;
    EXIT WHEN emp_cur%notfound;
    DBMS_OUTPUT.put_line(emp_rec.eid ||'    '||emp_rec.ename ||'    '||emp_rec.dno);
  END LOOP;
END;
/
```

PL/SQL – User Defined Records

```
DECLARE
   type student is record
        (rno varchar(50), name varchar(50), branch varchar(10));
   s1 student;
   s2 student;
BEGIN
   -- Student 1 specification
   s1.rno  := '1';
   s1.name  := 'Ajay';
   s1.branch  := 'CSE';
   -- Student 1 specification
   s2.rno  := '5';
   s2.name  := 'Aman';
   s2.branch  := 'IT';
   -- Print Student 1 record
   dbms_output.put_line('Student 1 Rollno : '|| s1.rno);
   dbms_output.put_line('Student 1 Name   : '|| s1.name);
   dbms_output.put_line('Student 1 Branch : '|| s1.branch);
   -- Print Student 2 record
   dbms_output.put_line('Student 2 Rollno : '|| s2.rno);
   dbms_output.put_line('Student 2 Name   : '|| s2.name);
   dbms_output.put_line('Student 2 Branch : '|| s2.branch);
END;
/
```

PL/SQL – Cursor

- A cursor is temporary work area created in system memory when a SQL statement is executed.

- Oracle creates a memory area, known as the context area, for processing an SQL statement, which contains all the information needed for processing the statement; for example, the number of rows processed, etc.

- A cursor is a pointer to this context area. PL/SQL controls the context area through a cursor. A cursor holds the rows (one or more) returned by a SQL statement.

There are two types of cursors −

- Implicit cursors
- Explicit cursors

PL/SQL –Implicit Cursor

- Implicit cursors are automatically created when an SQL statement is executed.
- Whenever a DML statement (INSERT, UPDATE and DELETE) is issued, an implicit cursor is associated with this statement.
- For INSERT operations, the cursor holds the data that needs to be inserted. For UPDATE and DELETE operations, the cursor identifies the rows that would be affected.

%FOUND	Returns TRUE if an INSERT, UPDATE, or DELETE statement affected one or more rows or a SELECT INTO statement returned one or more rows. Otherwise, it returns FALSE.
%NOTFOUND	The logical opposite of %FOUND. It returns TRUE if an INSERT, UPDATE, or DELETE statement affected no rows, or a SELECT INTO statement returned no rows. Otherwise, it returns FALSE.
%ISOPEN	Always returns FALSE for implicit cursors, because Oracle closes the SQL cursor automatically after executing its associated SQL statement.
%ROWCOUNT	Returns the number of rows affected by an INSERT, UPDATE, or DELETE statement, or returned by a SELECT INTO statement.

PL/SQL –Implicit Cursor Example-

- **Consider a relation customers--**
- The following program will update the table and increase the salary of each customer by 500 and use the SQL%ROWCOUNT attribute to determine the number of rows affected −

customer			
cid	**cname**	**age**	**salary**
1	Raman	32	25000
2	Gagan	36	27000
3	Ajay	35	32000

```
DECLARE
  total_rows number(2);

BEGIN
  UPDATE customer SET salary = salary + 500 where cid=6;

IF sql%notfound THEN
  dbms_output.put_line('no customers selected');

ELSIF sql%found THEN
  total_rows := sql%rowcount;
  dbms_output.put_line( total_rows || ' customers selected ');

END IF;
END;
/
```

PL/SQL –Explicit Cursor

- Explicit cursors are user-defined cursors for gaining more control over the context area.
- An explicit cursor should be defined in the declaration section of the PL/SQL Block.
- It is created on a SELECT Statement which returns more than one row.

➢ **The syntax for creating an explicit cursor is −**

CURSOR cursor_name IS select_statement;

➢ **Working with an explicit cursor includes the following steps −**

- ❑ Declaring the cursor for initializing the memory
- ❑ Opening the cursor for allocating the memory
- ❑ Fetching the cursor for retrieving the data
- ❑ Closing the cursor to release the allocated memory

PL/SQL –Explicit Cursor

- Declaring the cursor for initializing the memory

```
CURSOR c_customers IS SELECT id, name, address FROM customers;
```

- Opening the cursor for allocating the memory

```
OPEN c_customers;
```

- Fetching the cursor for retrieving the data

```
FETCH c_customers INTO c_id, c_name, c_addr;
```

- Closing the cursor to release the allocated memory

```
CLOSE c_customers;
```

PL/SQL –Explicit Cursor Example-

```
DECLARE
   v_cid customer.cid%type;
   v_cname customer.cname%type;
   v_age customer.age%type;
   v_salary customer.salary%type;

CURSOR cur_customer is
      SELECT cid, cname, age, salary FROM customer;

BEGIN
   OPEN cur_customer;

 LOOP
   FETCH cur_customer into v_cid, v_cname, v_age, v_salary;
      EXIT WHEN cur_customer%notfound;
      dbms_output.put_line(v_cid||'  '||v_cname||v_age ||'  '||v_salary);

END LOOP;
   CLOSE cur_customer;
END;
/
```

PL/SQL – Cursor Cont..

Program 13 : Display name, hire date of all employees using cursors.

Consider a relation- **emp1**

SSN	NAME	HDATE	SALARY
1	Amit	01-JAN-20	35000
2	Aman	01-MAR-20	34000
3	Sunil	01-APR-20	37000
4	Sumit	01-MAY-20	42000
5	Ravi	01-JUN-20	40000
6	Raman	05-NOV-20	43000
7	Rahul	03-NOV-20	41000
8	Vikas	03-DEC-20	39000

PL/SQL – Cursor Cont..

Program 13 : Display name, hire date of all employees using cursors.

Consider a relation- **emp1**

```
declare
cursor c1 is
select name, hdate from emp1;
ename emp1.name%type;
hiredate emp1.hdate%type;
begin
open c1;
loop
fetch c1 into ename, hiredate;
exit when c1%notfound;
dbms_output.put_line(ename||hiredate);
end loop;
close c1;
end;
/
```

SSN	NAME	HDATE	SALARY
1	Amit	01-JAN-20	35000
2	Aman	01-MAR-20	34000
3	Sunil	01-APR-20	37000
4	Sumit	01-MAY-20	42000
5	Ravi	01-JUN-20	40000
6	Raman	05-NOV-20	43000
7	Rahul	03-NOV-20	41000
8	Vikas	03-DEC-20	39000

PL/SQL – Cursor Cont..

Program 14: Display details of first 5 highly paid employees using cursors.

Consider a relation- **emp1**

SSN	NAME	HDATE	SALARY
1	Amit	01-JAN-20	35000
2	Aman	01-MAR-20	34000
3	Sunil	01-APR-20	37000
4	Sumit	01-MAY-20	42000
5	Ravi	01-JUN-20	40000
6	Raman	05-NOV-20	43000
7	Rahul	03-NOV-20	41000
8	Vikas	03-DEC-20	39000

PL/SQL – Cursor Cont..

Program 14: Display details of first 5 highly paid employees using cursors.

Consider a relation- **emp1**

SSN	NAME	HDATE	SALARY
1	Amit	01-JAN-20	35000
2	Aman	01-MAR-20	34000
3	Sunil	01-APR-20	37000
4	Sumit	01-MAY-20	42000
5	Ravi	01-JUN-20	40000
6	Raman	05-NOV-20	43000
7	Rahul	03-NOV-20	41000
8	Vikas	03-DEC-20	39000

```
declare
cursor emp_cur is
select * from emp1 order by salary desc;
emp_row emp_cur %rowtype;
begin
open emp_cur;
loop
fetch emp_cur into emp_row;
exit when emp_cur %rowcount>5;
dbms_output.put_line (emp_row.ssn||'  '||emp_row.name||emp_row.hdate||'
'||emp_row.salary);
end loop;
dbms_output.put_line ('Total rows selected:'||emp_cur%rowcount);
close emp_cur;
end;
/
```

PL/SQL – Procedure

- A stored procedure or procedure is a PL/SQL block that performs one or more specific tasks.
- PL/SQL subprograms are named PL/SQL blocks that can be invoked with a set of parameters. PL/SQL provides two kinds of subprograms −

  - **Functions** − These subprograms return a single value; mainly used to compute and return a value.
  - **Procedures** − These subprograms do not return a value directly; mainly used to perform an action.

General Syntax for procedure:

```
create [or replace] procedure procedure_name
[(parameter_name [in | out | in out] type [, ...])]
{is | as}
    < declaration section >
begin
    < execution section >
exception
    < exception section>
end procedure_name;
```

PL/SQL – Procedure Example

The following example creates a simple procedure that displays the string 'Hello World!' on the screen when executed.

```
CREATE OR REPLACE PROCEDURE greetings
AS
BEGIN
   dbms_output.put_line('Hello World!');
END;
/
```

The above procedure named 'greetings' can be called with the EXECUTE keyword as −

```
EXECUTE greetings;
```

The above call will display −

```
Hello World
PL/SQL procedure successfully completed.
```

PL/SQL – Parameters in Procedure

The following table lists out the parameter modes in PL/SQL subprograms −

IN
An IN parameter lets you pass a value to the subprogram. It is a read-only parameter. Inside the subprogram, an IN parameter acts like a constant.

OUT
An OUT parameter returns a value to the calling program. Inside the subprogram, an OUT parameter acts like a variable. You can change its value and reference the value after assigning it.

IN OUT
An IN OUT parameter passes an initial value to a subprogram and returns an updated value to the caller. It can be assigned a value and the value can be read.

PL/SQL –Procedure Example 1- (in, out parameter)

This program finds the minimum of two values. Here, the procedure takes two numbers using the IN mode and returns their minimum using the OUT parameters.

```
declare
   a number;
   b number;
   c number;
procedure findmin(x in number, y in number, z out number) is
begin
   if x < y then
      z:= x;
   else
      z:= y;
   end if;
end;
begin
   a:= &a;
   b:= &b;
   findmin(a, b, c);
   dbms_output.put_line(' minimum number is  : ' || c);
end;
/
```

PL/SQL –Procedure Example 2- (in out parameter)

This procedure computes the square of value of a passed value. This example shows how we can use the same parameter to accept a value and then return another result.

```
declare
   a number;
procedure squarenum(x in out number) is
begin
  x := x * x;
end;
begin
   a:= &a;
   squarenum(a);
   dbms_output.put_line(' square of given number is: ' || a);
end;
/
```

PL/SQL –Functions

The simplified syntax for the CREATE OR REPLACE PROCEDURE statement is as follows

```
CREATE [OR REPLACE] FUNCTION function_name
[(parameter_name [IN | OUT | IN OUT] type [, ...])]
RETURN return_datatype
{IS | AS}
BEGIN
   < function_body >
END [function_name];
```

PL/SQL –Functions Example 1-

Creating a Function

```
create or replace function total_emp
return number is
   total number(2) := 0;
begin
   select count(*) into total from emp;
   return total;
end;
/
```

SSN	NAME	HDATE	SALARY
1	Amit	01-JAN-20	35000
2	Aman	01-MAR-20	34000
3	Sunil	01-APR-20	37000
4	Sumit	01-MAY-20	42000
5	Ravi	01-JUN-20	40000
6	Raman	05-NOV-20	43000
7	Rahul	03-NOV-20	41000
8	Vikas	03-DEC-20	39000

Calling a Function

```
DECLARE
   c number(2);
BEGIN
   c := total_emp();
   dbms_output.put_line('Total no. of Customers: ' || c);
END;
/
```

PL/SQL –Functions Example 2-

The following example demonstrates Declaring, Defining, and Invoking a Simple PL/SQL Function that computes and returns the maximum of two values.

```
declare
   a number;
   b number;
   c number;
function findmax(x in number, y in number)
return number is
    z number;
begin
   if x > y then
      z:= x;
   else
      z:= y;
   end if;
   return z;
end;
begin
   a:= &a;
   b:= &b;
   c := findmax(a, b);
   dbms_output.put_line(' maximum number is: ' || c);
end;
/
```

PL/SQL –Functions Example 3-

The following program calculates the factorial of a given number by calling itself recursively

```
declare
   num number;
   factorial number;
function fact(x number)
return number is
   f number;
begin
   if x=0 then
      f := 1;
   else
      f := x * fact(x-1);
   end if;
return f;
end;
begin
   num:= &num;
   factorial := fact(num);
   dbms_output.put_line(' factorial '|| num || ' is ' || factorial);
end;
/
```

PL/SQL –Functions Program-12

Write a PL/SQL block to delete a record. If delete is successful return 1 else return 0.

Creating Function

Calling Function

PL/SQL –Functions Program-12

Write a PL/SQL block to delete a record. If delete is successful return 1 else return 0.

Creating Function

```
create or replace function fun1 (n customer.cid%type)
return number is
a number;
begin
delete from customer where cid=n;
if sql %found then
return 1;
else
return 0;
end if;
end;
/
```

Calling Function

```
declare
n number;
begin
n:=fun1(&cid);
dbms_output.put_line(n);
if n=0 then
dbms_output.put_line('deletion unsuccessfully');
else
if n=1 then
dbms_output.put_line('deletion successful');
end if;
end if;
end;
/
```

PL/SQL –Triggers

- Triggers are stored programs, which are invoked by Oracle engine automatically whenever a specified event occurs.
- Trigger is stored into database and invoked repeatedly, when specific condition match.
- Triggers are written to be executed in response to any of the following events −
    - A database manipulation (DML) statement (DELETE, INSERT, or UPDATE)
    - A database definition (DDL) statement (CREATE, ALTER, or DROP).
    - A database operation (SERVERERROR, LOGON, LOGOFF, STARTUP or SHUTDOWN).

Benefits of Triggers-

- Generating some derived column values automatically
- Enforcing referential integrity
- Event logging and storing information on table access
- Auditing
- Imposing security authorizations
- Preventing invalid transactions

PL/SQL –Triggers

The syntax for creating a trigger is −

```
CREATE [OR REPLACE ] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF }
{INSERT [OR] | UPDATE [OR] | DELETE}
 [OF col_name]
 ON table_name
 [REFERENCING OLD AS o NEW AS n]
 [FOR EACH ROW]
 WHEN (condition)

 DECLARE
   Declaration-statements
 BEGIN
   Executable-statements
 EXCEPTION
   Exception-handling-statements
 END;
```

PL/SQL –Trigger Example 1

WRITE A DATA BASE TRIGGER, WHICH SHOULD NOT DELETE FROM PRODUCT TABLE IF THE DAY IS SUNDAY.

```
create or replace trigger
check_day
before delete on customer
begin
if to_char(sysdate,'day')=Monday' then
raise_application_error(-20001,'to day is monday');
end if;
end;
/
```

```
create or replace trigger
check_day
before delete on customer
begin
if to_char(sysdate,'d')=2 then
raise_application_error(-20001,'today is Monday, you can`t delete');
end if;
end;
```

PL/SQL –Trigger Example 2

WRITE A DATABASE TRIGGER WHICH FIRES IF YOU TRY TO INSERT, UPDATE, OR DELETE AFTER 7'O' CLOCK

```
create or replace trigger gettime before
insert or update or delete on customer for each row

declare
a varchar2 (10);
begin
select to_char (sysdate,'HH24:MI:SS') into a from dual;
if a > '11:45:00'
then
raise_application_error (-20500,'you can not do this operation now');
end if;
end;
```

PL/SQL –Trigger Example 3

WRITE A DATA BASE TRIGGER, WHICH ACTS JUST LIKE PRIMARY KEY AND DOES NOT ALLOW DUPLICATE

```
create or replace trigger prikey before
insert on test for each row

declare
a number;
begin
if :new.id is null then
raise_application_error(-20001, 'id can not be null');
end if;
select count(*) into a from test where id=:new.id;
if a >0 then
raise_application_error(-20001,'unique key rule is voilated');
end if;
end;
/
```

PL/SQL –Trigger Example 4

Write a PL/SQL for displaying salary changes in emp1 table.

```
create or replace trigger sal_changes
before delete or insert or update on customer
for each row
when (new.cid > 0)
declare
   sal_diff number;
begin
   sal_diff := :new.salary  - :old.salary;
   dbms_output.put_line('old salary: ' || :old.salary);
   dbms_output.put_line('new salary: ' || :new.salary);
   dbms_output.put_line('salary difference: ' || sal_diff);
end;
/
```

SSN	NAME	HDATE	SALARY
1	Amit	01-JAN-20	35000
2	Aman	01-MAR-20	34000
3	Sunil	01-APR-20	37000
4	Sumit	01-MAY-20	42000
5	Ravi	01-JUN-20	40000
6	Raman	05-NOV-20	43000
7	Rahul	03-NOV-20	41000
8	Vikas	03-DEC-20	39000

insert into emp1 values (9, 'kriti', ‘01-dec-2020', 7500.00);
old salary:
new salary: 7500
salary difference:

PL/SQL –Exception handling

```
DECLARE
a int:=10;
b int:=0;
answer int;

BEGIN
answer:=a/b;
dbms_output.put_line('the result after division is'||answer);

exception
WHEN zero_divide THEN
        dbms_output.put_line('dividing by zero please check the values again');
        dbms_output.put_line('the value of a is '||a);
        dbms_output.put_line('the value of b is '||b);
END;
```

PL/SQL –Exception handling

NO_DATA_FOUND: It is raised WHEN a SELECT INTO statement returns no rows.
For eg:

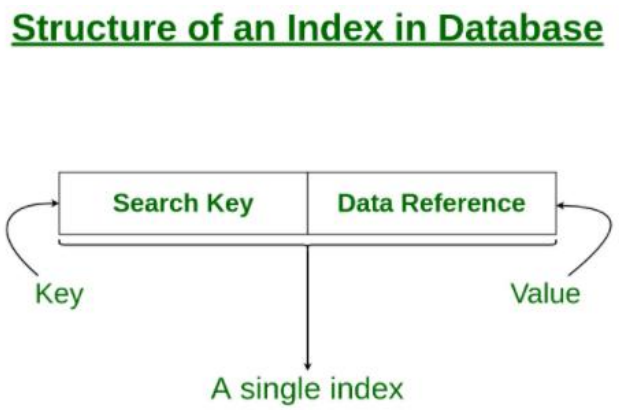
```
DECLARE
temp varchar(20);

BEGIN
SELECT g_id into temp from geeks where g_name=‘Aman';

exception
WHEN no_data_found THEN
        dbms_output.put_line('ERROR');
        dbms_output.put_line('there is no name as Aman');
end;
```

Indexes

- A **database index** is a data structure that improves the speed of data retrieval operations on a database table.
- Indexing is used to optimize the performance of a database by minimizing the number of disk accesses required when a query is processed.



- The first column is the Search key that contains a copy of the primary key or candidate key of the table. These values are stored in sorted order so that the corresponding data can be accessed quickly.
- The second column is the Data Reference or Pointer which contains the address of the disk block where that particular key value can be found.

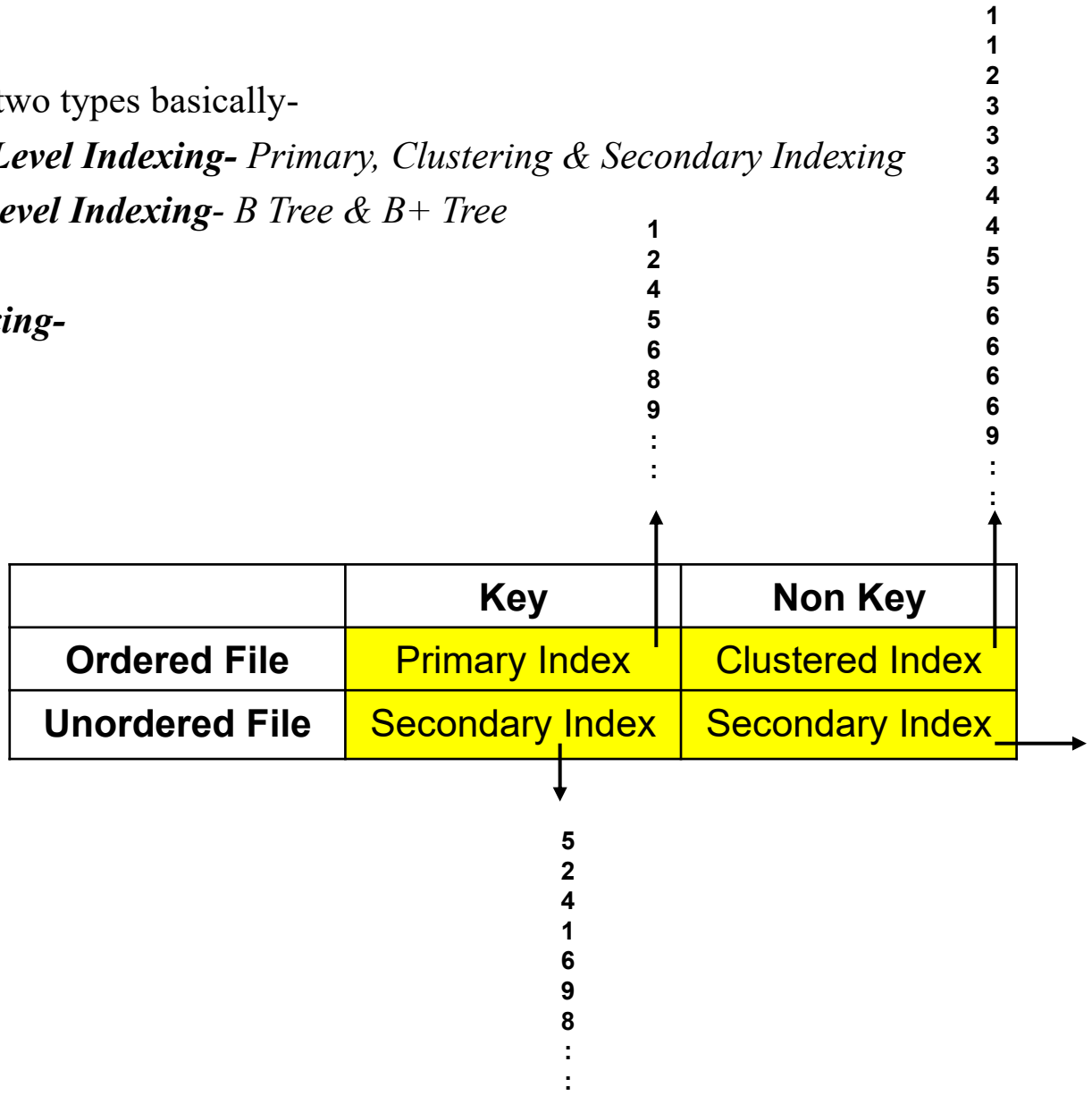
Indexes

- Indexing of two types basically-

  Single Level Indexing- *Primary, Clustering & Secondary Indexing*

  Multi Level Indexing- *B Tree & B+ Tree*

Single Level Indexing-

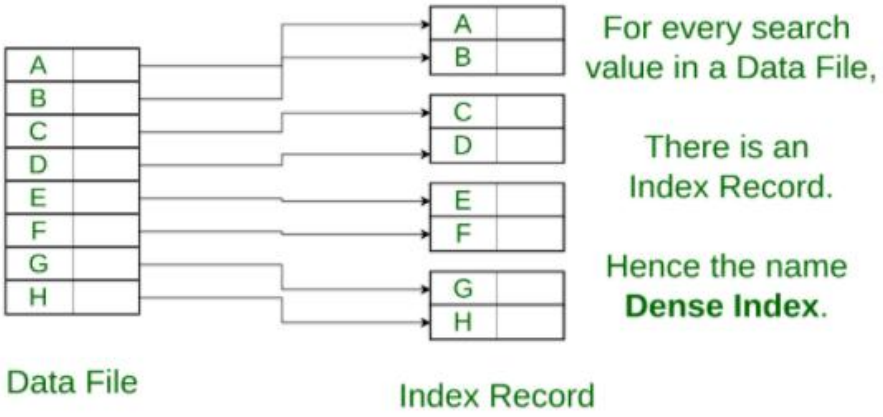


Primary Index

- If the index is created on the basis of the primary key of the table, then it is known as primary indexing. These primary keys are unique to each record and contain 1:1 relation between the records.
- As primary keys are stored in sorted order, the performance of the searching operation is quite efficient.
- The primary index can be classified into two types: Dense index and Sparse index.

Dense Index:

- For every search key value in the data file, there is an index record.
- This record contains the search key and also a reference to the first data record with that search key value.



Primary Index

Sparse Index:

- The index record appears only for a few items in the data file. Each item points to a block as shown.
- To locate a record, we find the index record with the largest search key value less than or equal to the search key value we are looking for.
- We start at that record pointed by the index record and proceed along with the pointers in the file (that is, sequentially) until we find the desired record.

B1	1 Ram 21 Indore 2 Aman 23 Bhopal 3 Amit 22 Ujjain 4 Anuj 21 Indore
B2	5 ..... 6 ..... 7 ..... 8 .....
B3	9 ..... 10 ..... 11 ..... 12 .....
B4	13 ..... 14 ..... 15 ..... 16 .....
:	

Hard disk

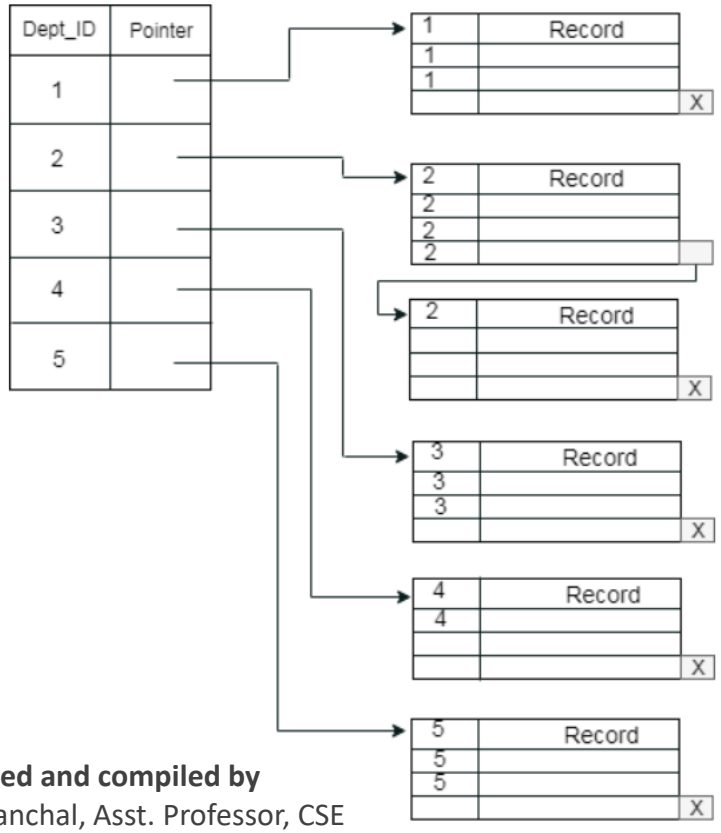
Key (Roll No)	Pointer
1	B1
5	B2
9	B3
13	B4

Index Table

Clustering Index

- A clustered index can be defined as an ordered data file. Sometimes the index is created on non-primary key columns which may not be unique for each record.
- In this case, to identify the record faster, we will group two or more columns to get the unique value and create index out of them. This method is called as clustering index.
- The records which have similar characteristics are grouped, and indexes are created for these group.

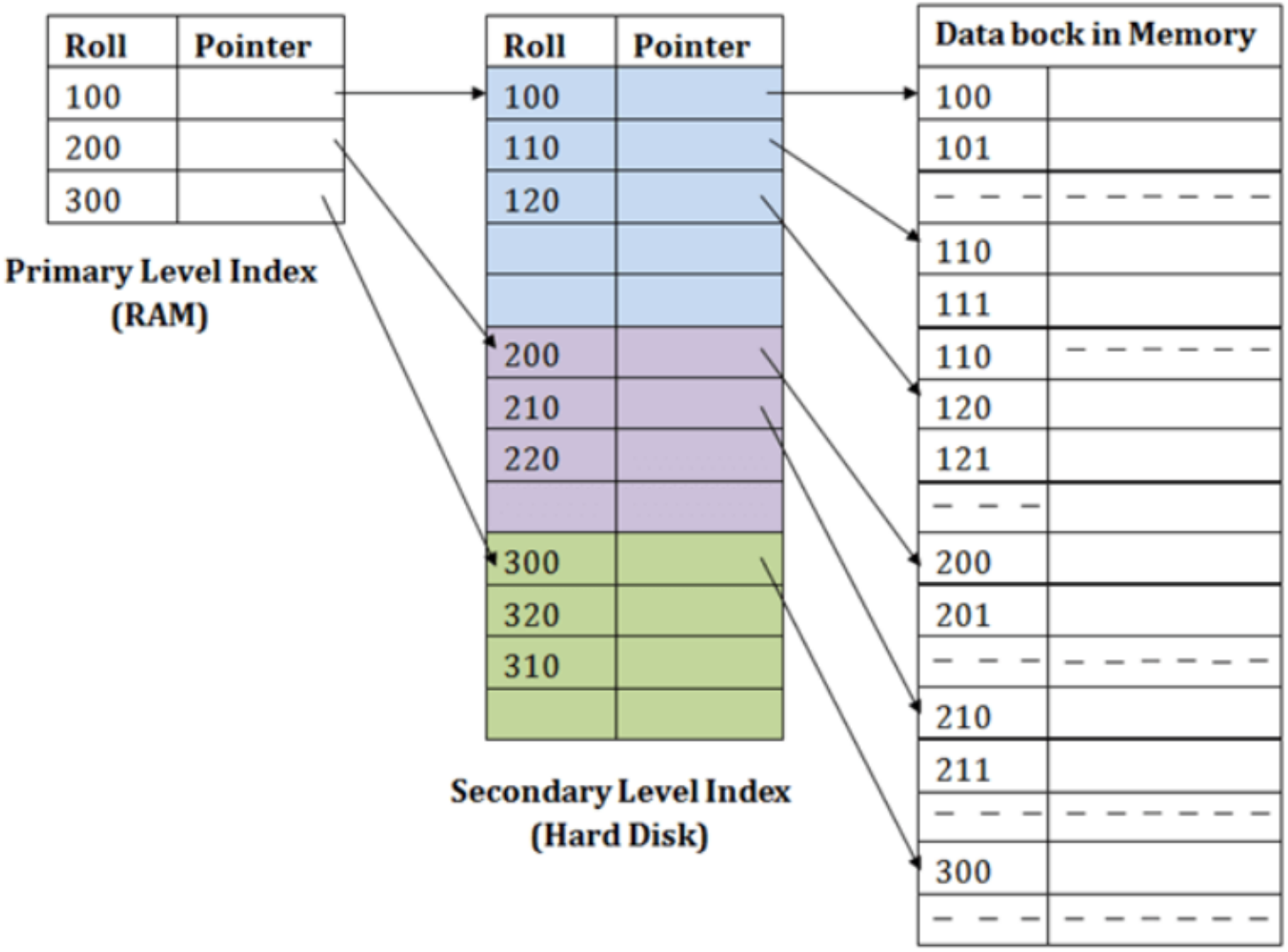
- **Example:** suppose a company contains several employees in each department.
- Suppose we use a clustering index, where all employees which belong to the same Dept_ID are considered within a single cluster, and index pointers point to the cluster as a whole.
- Here Dept_Id is a non-unique key.



Non-clustered or Secondary Indexing

- A secondary index just tells us where the data lies, i.e. it gives us a list of virtual pointers or references to the location where the data is actually stored.
- In secondary indexing, to reduce the size of mapping, another level of indexing is introduced.
- In this method, the huge range for the columns is selected initially so that the mapping size of the first level becomes small. Then each range is further divided into smaller ranges.
- The mapping of the first level is stored in the primary memory, so that address fetch is faster. The mapping of the second level and actual data are stored in the secondary memory (hard disk).
- It requires more time as compared to the clustered index because some amount of extra work is done in order to extract the data by further following the pointer. In the case of a clustered index, data is directly present in front of the index.

Non-clustered or Secondary Indexing



B Tree

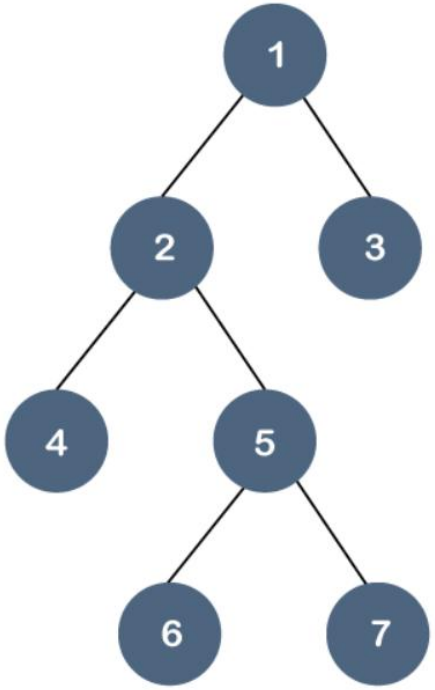
- B tree is a self-balancing tree, and it is a m-way tree where m defines the order of the tree.
- B tree is a generalization of the Binary Search tree in which a node can have more than one key and more than two children depending upon the value of m.
- In the B tree, the data is specified in a sorted order having lower values on the left subtree and higher values in the right subtree.

The following are the properties of the B tree:

- In the B tree, all the leaf nodes must be at the same level, whereas, in the case of a binary tree, the leaf nodes can be at different levels.

B Tree

- Let's understand this property through an example.



In this tree, all the leaf nodes are not at the same level, but they have the utmost two children.
Therefore, we can say that the above tree is a binary tree but not a B tree.

B Tree

- If the B tree has an order of m, then each node can have a maximum of m children.
- In the case of minimum children, the leaf nodes have zero children, the root node has two children, and the internal nodes have a ceiling of m/2.
- Each node can have maximum (m-1) keys. For example, if the value of m is 5 then the maximum value of keys is 4.
- If we perform insertion in the B tree, then the node is always inserted in the leaf node.

Suppose we want to create a B tree of order 3 by inserting values from 1 to 10.

Step 1: First, we create a node with 1 value as shown below:



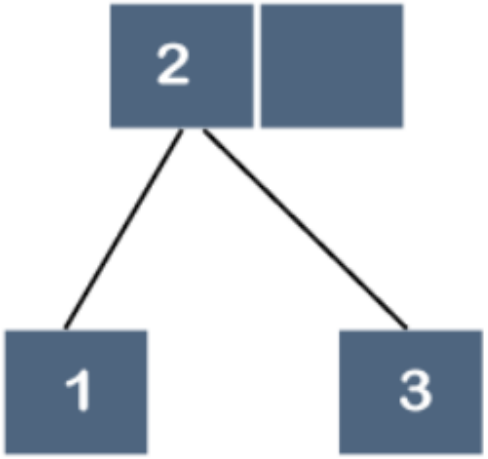
Step 2: The next element is 2, which comes after 1 as shown below:



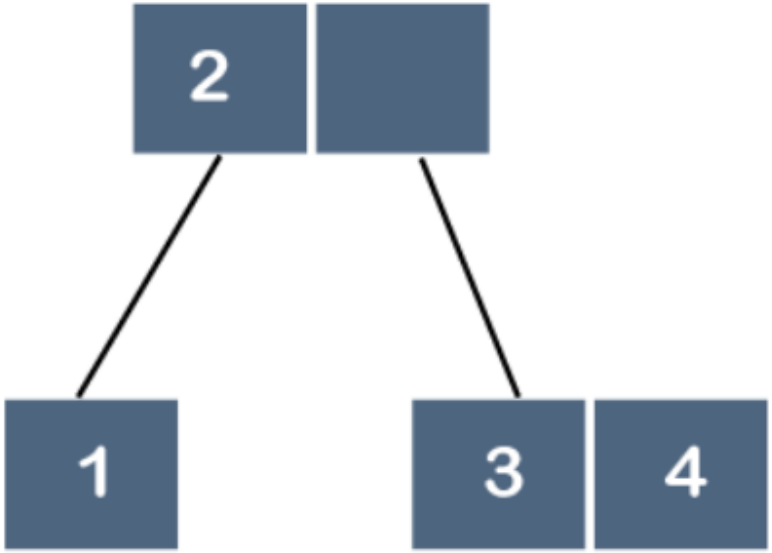
Step 3: The next element is 3, and it is inserted after 2 as shown below:



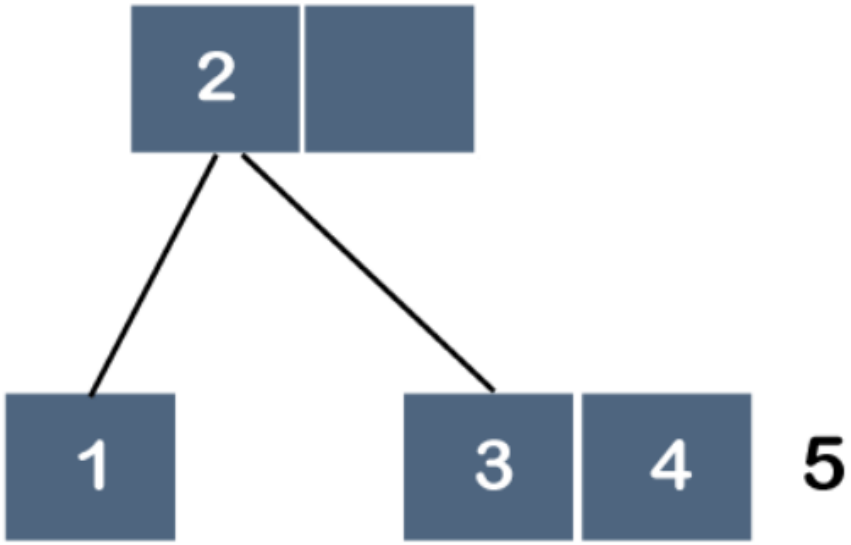
As we know that each node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 2, so it moves to its parent. The node 2 does not have any parent, so it will become the root node as shown in next:



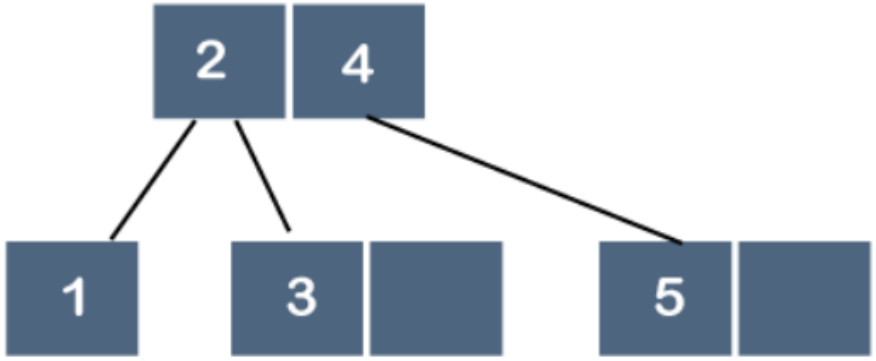
Step 4: The next element is 4. Since 4 is greater than 2 and 3, so it will be added after the 3 as shown below:



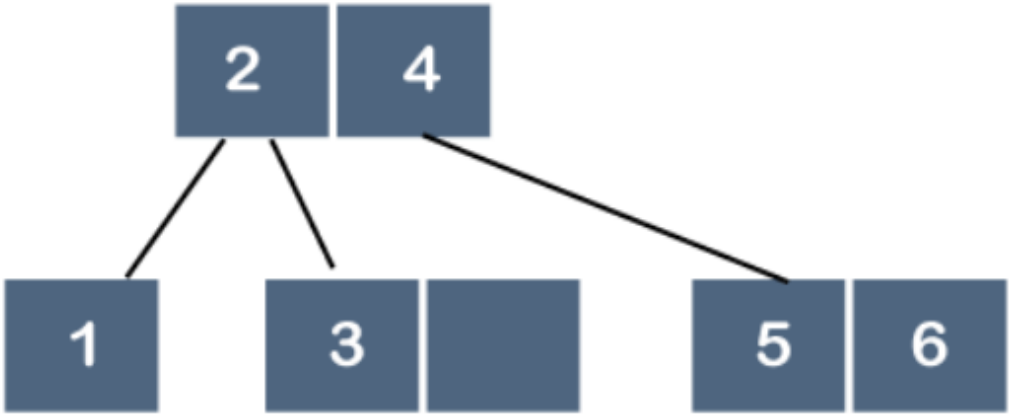
Step 5: The next element is 5. Since 5 is greater than 2, 3 and 4 so it will be added after 4 as shown below:



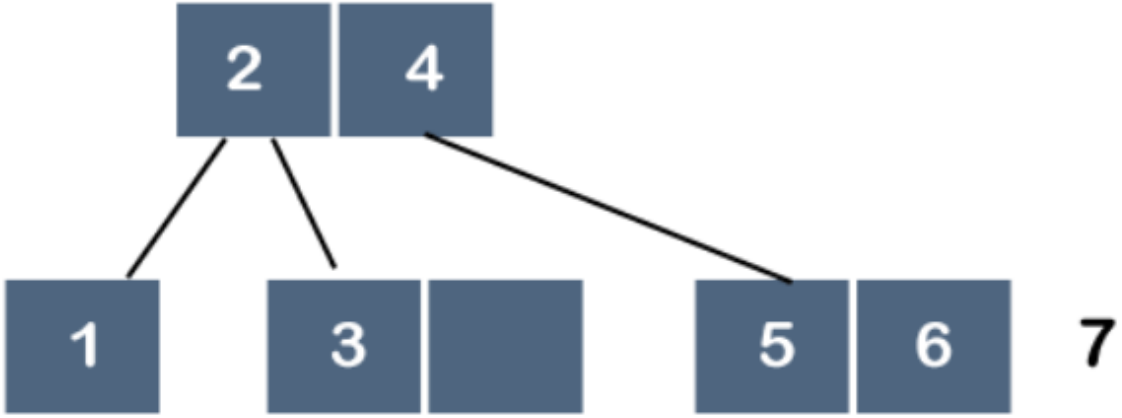
As we know that each node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 4, so it moves to its parent. The parent is node 2; therefore, 4 will be added after 2 as shown below:



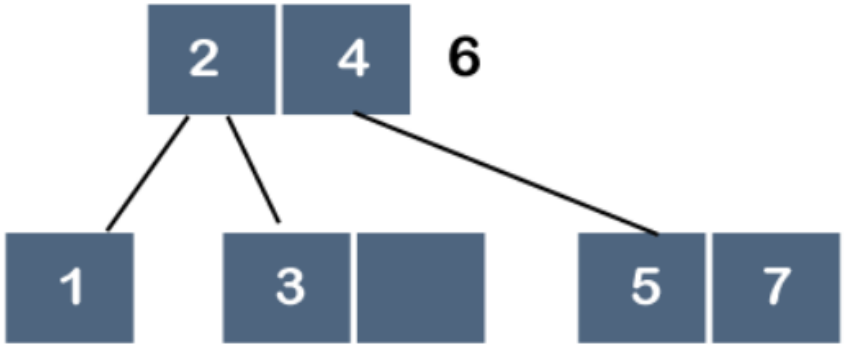
Step 6: The next element is 6. Since 6 is greater than 2, 4 and 5, so 6 will come after 5 as shown below:



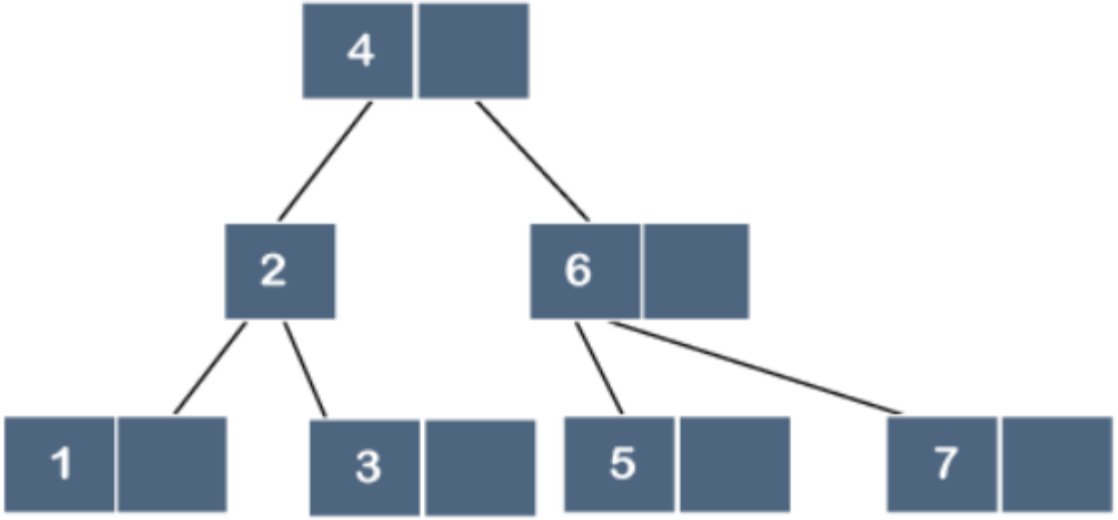
Step 7: The next element is 7. Since 7 is greater than 2, 4, 5 and 6, so 7 will come after 6 as shown below:



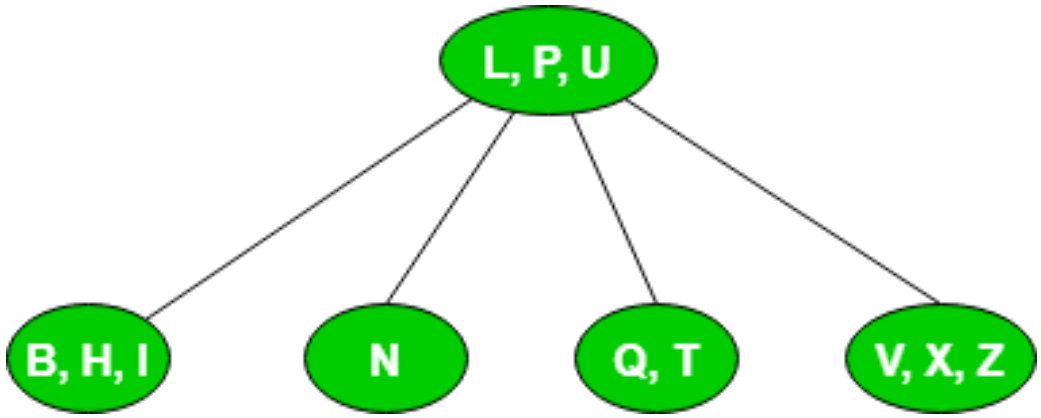
As we know that each node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 6, so it moves to its parent as shown below:



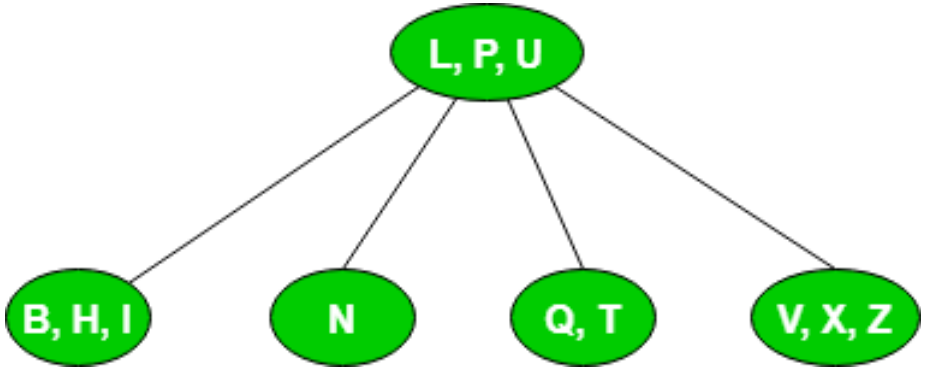
But, 6 cannot be added after 4 because the node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 4, so it moves to its parent. As node 4 does not have any parent, node 4 will become a root node as shown below:

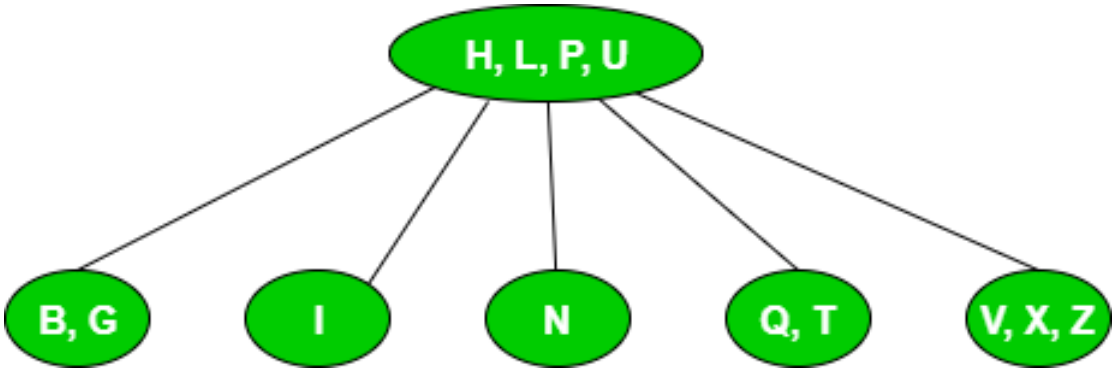


Que - 2. Consider the following 2-3-4 tree (i.e., B-tree with a minimum degree of two) in which each data item is a letter. The usual alphabetical ordering of letters is used in constructing the tree.



What is the result of inserting G in the above tree?





Now root node with keys H, L, P, U is overflowed which leads to splitting of root node into two parts HL and U and middle element P will be root node which matches option B

B+ Tree

- The B+ tree is also known as an advanced self-balanced tree because every path from the root of the tree to the leaf of the tree has the same length.
- Here, the same length means that all the leaf nodes occur at the same level. It will not happen that some of the leaf nodes occur at the third level and some of them at the second level.
- A B+ tree index is considered a multi-level index, but the B+ tree structure is not similar to the multi-level index sequential files.

Why is the B+ tree used?

- A B+ tree is used to store the records very efficiently by storing the records in an indexed manner using the B+ tree indexed structure.
- Due to the multi-level indexing, the data accessing becomes faster and easier.

B+ Tree

B+ tree Node Structure

- The node structure of the B+ tree contains pointers and key values shown in the below figure:

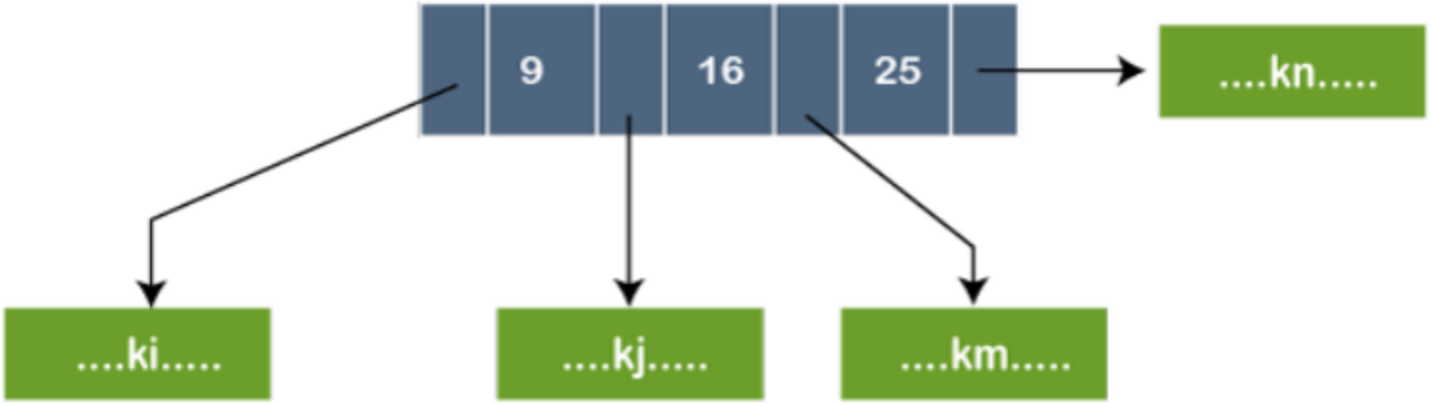


- As we can observe in the above B+ tree node structure that it contains n-1 key values (K_1 to K_{n-1}) and n pointers (P_1 to P_n).

- The search key values which are placed in the node are kept in sorted order. Thus, if i<j then ki<kj.

B+ Tree

Example of B+ tree



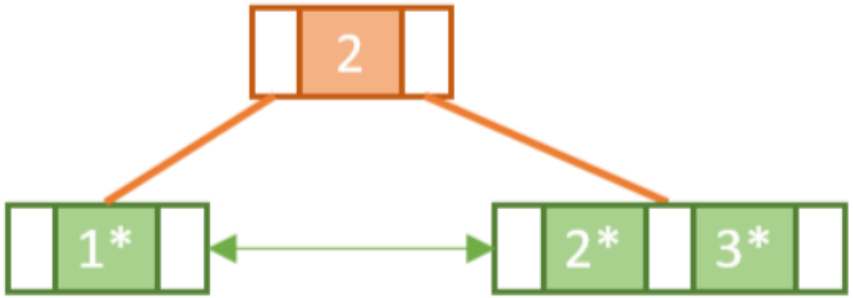
- In the above figure, the node contains three key values, i.e., 9, 16, and 25.
- The pointer that appears before 9, contains the key values less than 9 represented by ki.
- The pointer that appears before 16, contains the key values greater than or equal to 9 but less than 16 represented by kj.
- The pointer that appears before 25, contains the key values greater than or equal to 16 but less than 25 represented by kn.

To start with, we'll insert Keys 1 and 2 into the root node in ascending order:

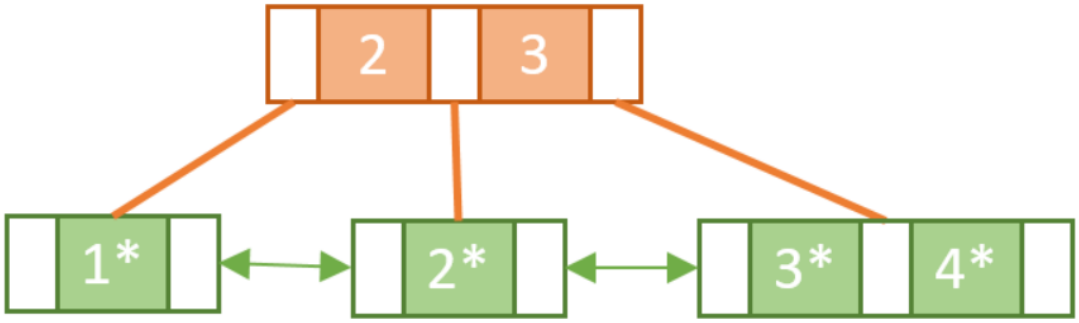


When we come to insert Key 3, we find that in doing so we will exceed the capacity of the root node.

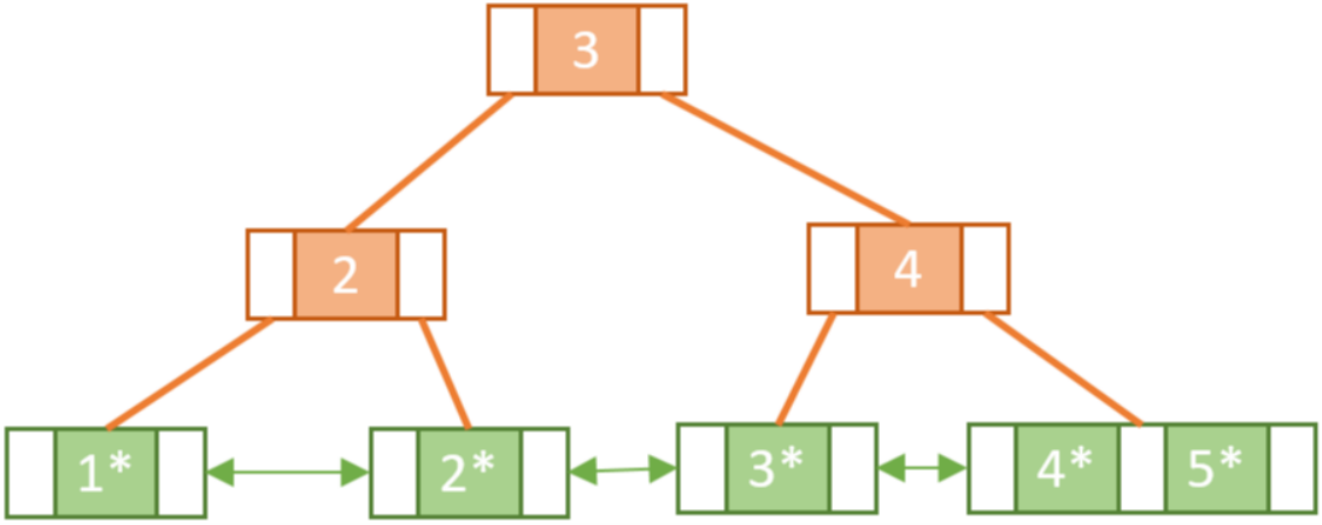
Similar to a normal B-tree this means we need to perform a split operation. However, unlike with the B-tree, we must copy-up the first key in the new rightmost leaf node. As mentioned, this is so we can make sure we have a key/value pair for Key 2 in the leaf nodes:



Next, we add Key 4 to the rightmost leaf node. Since it's full, we need to perform another a split operation and copy-up Key 3 to the root node:

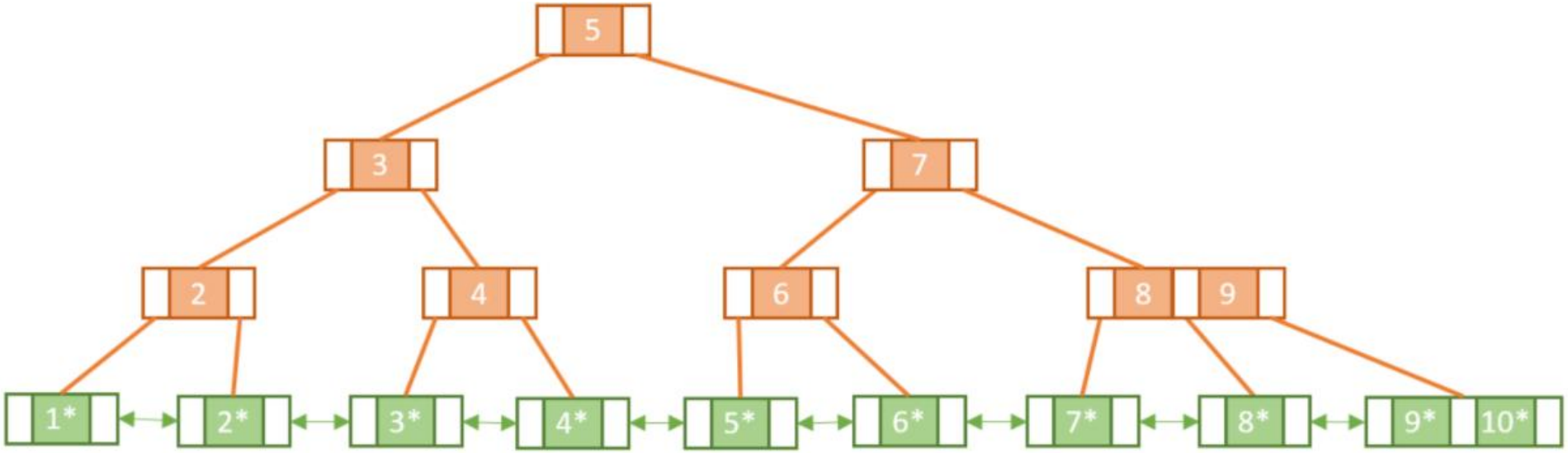


Now, let's add 5 to the rightmost leaf node. Once again to keep the order, we'll split the leaf node and copy-up 4. Since that will overflow the root node, we'll have to perform another split operation splitting the root node into two nodes and promoting 3 into a new root node:



Notice the difference between splitting a leaf node and splitting an internal node. When we split the internal node in the second split operation we didn't copy-up Key 3.

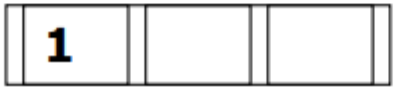
In the same way, we keep adding the keys from 6 to 10, each time splitting and copying-up when necessary until we have reached our final tree:



- Suppose each B+-tree node can hold up to 4 pointers and 3 keys.
- m=3 (odd), d=1
- Half-full (for odd m value)
  - Leaf node, at least 2 (d+1) entries
  - Non-leaf nodes, at least 2 (d+1) pointers (1 entry)
- Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10

Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10

- Insert 1



Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



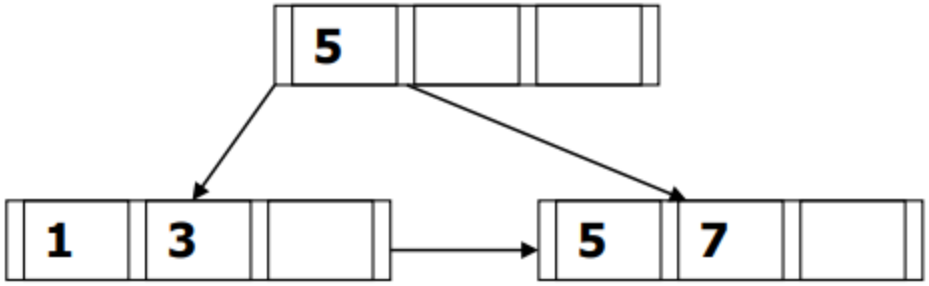
- Insert 3, 5



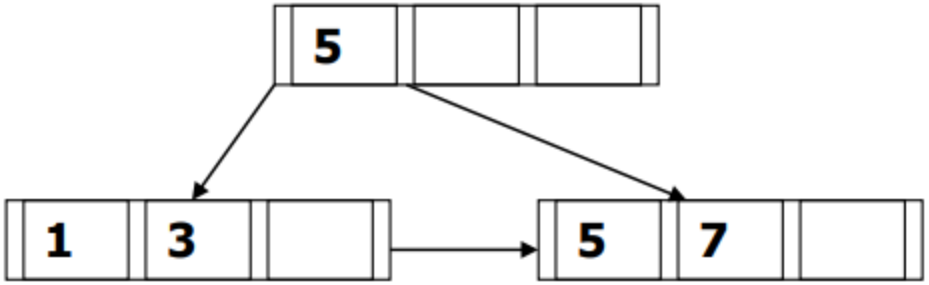
Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



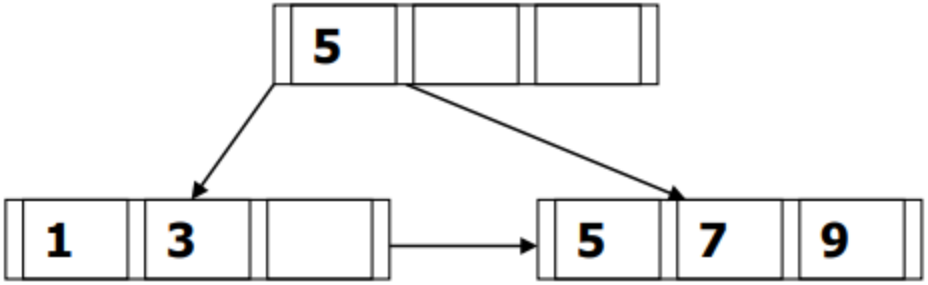
- Insert 7



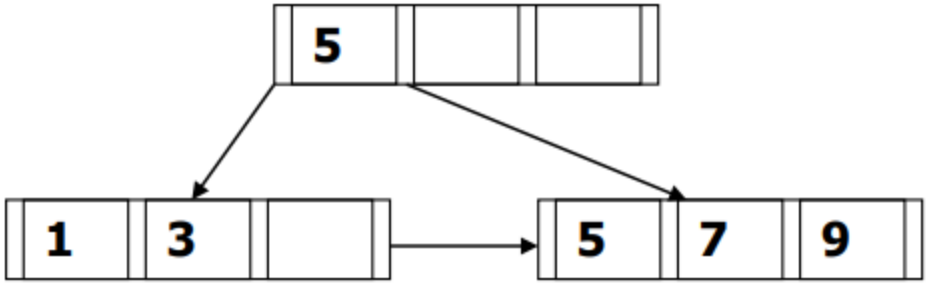
Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



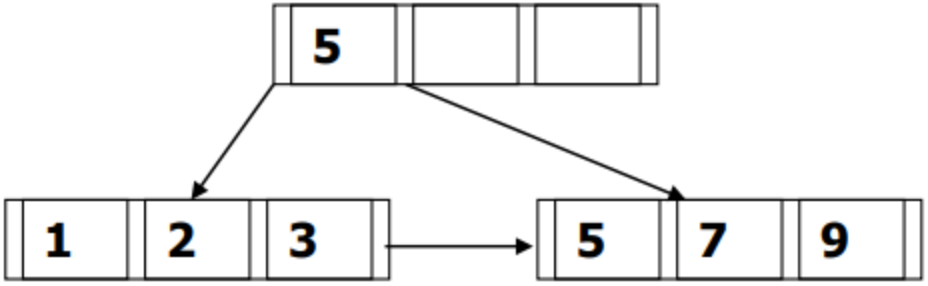
- Insert 9



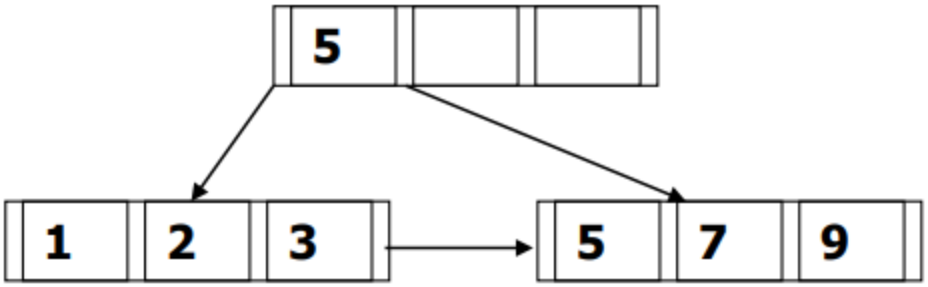
Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



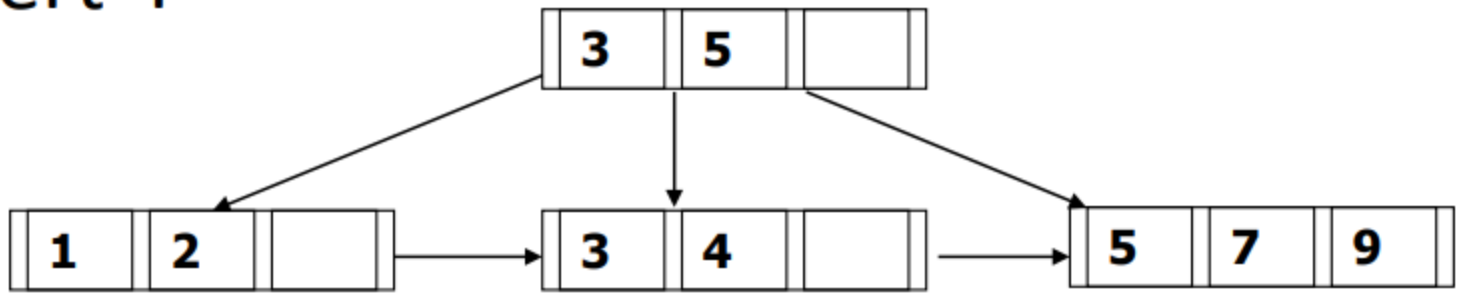
- Insert 2



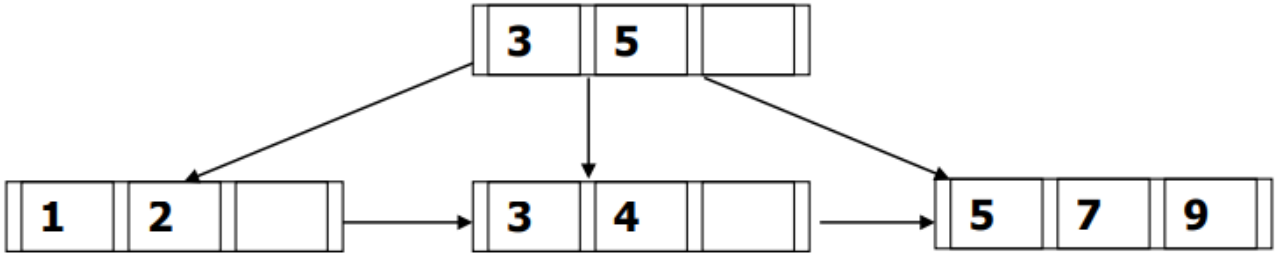
Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



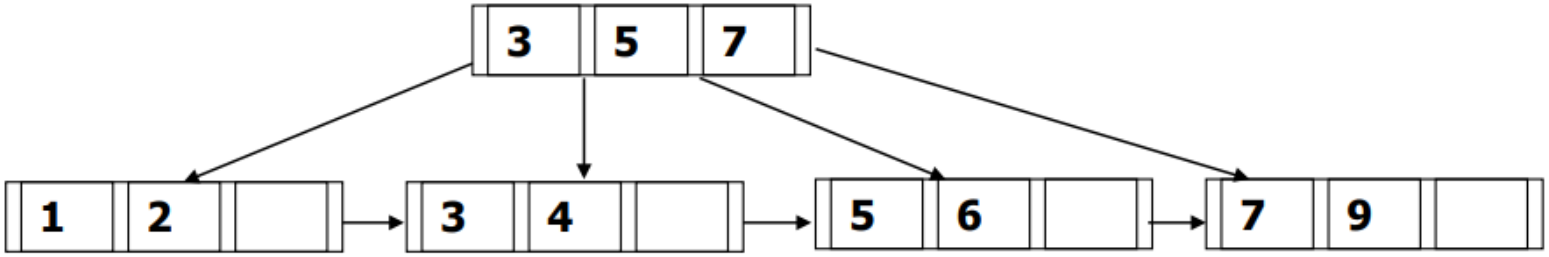
- Insert 4



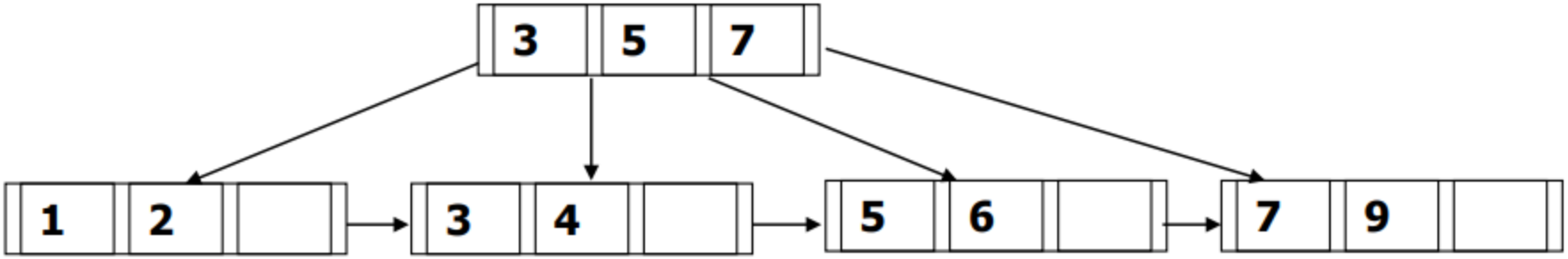
Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



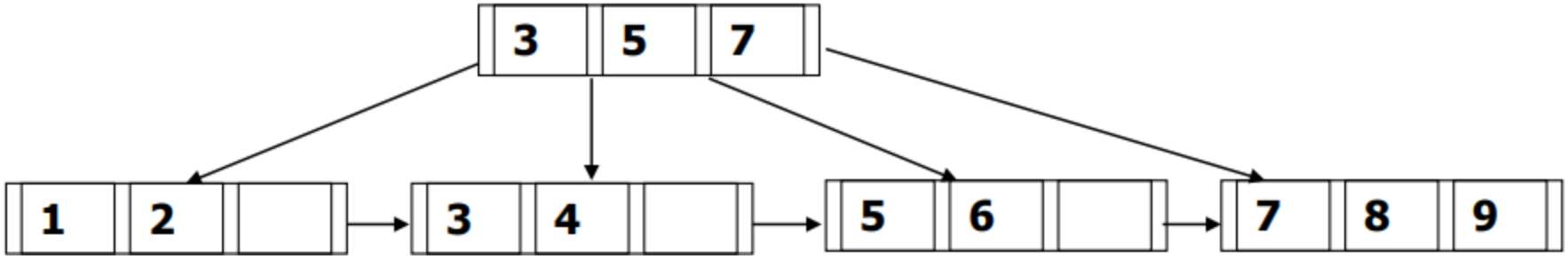
- Insert 6



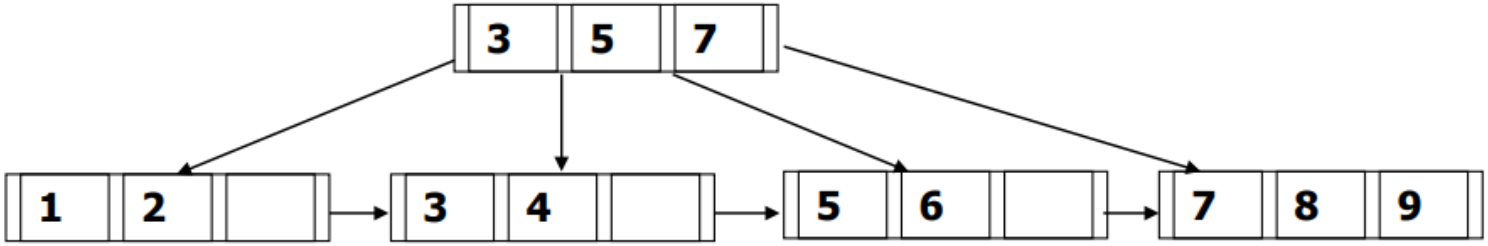
Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



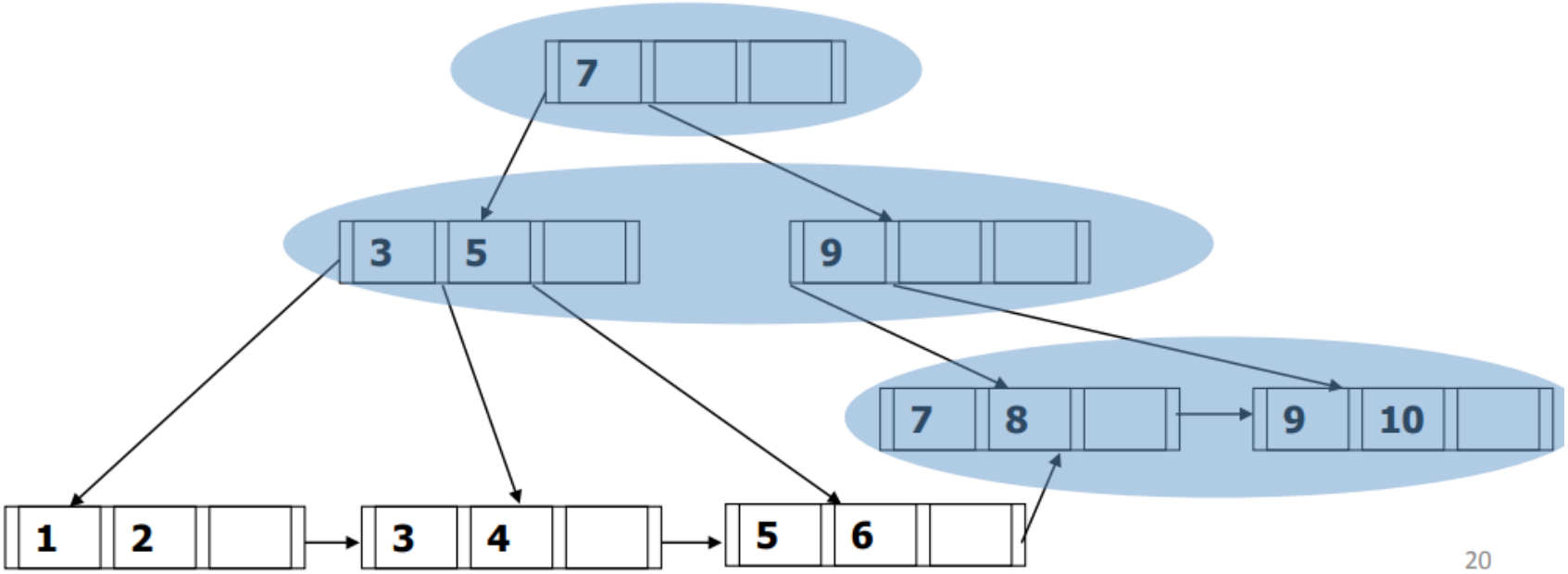
- Insert 8



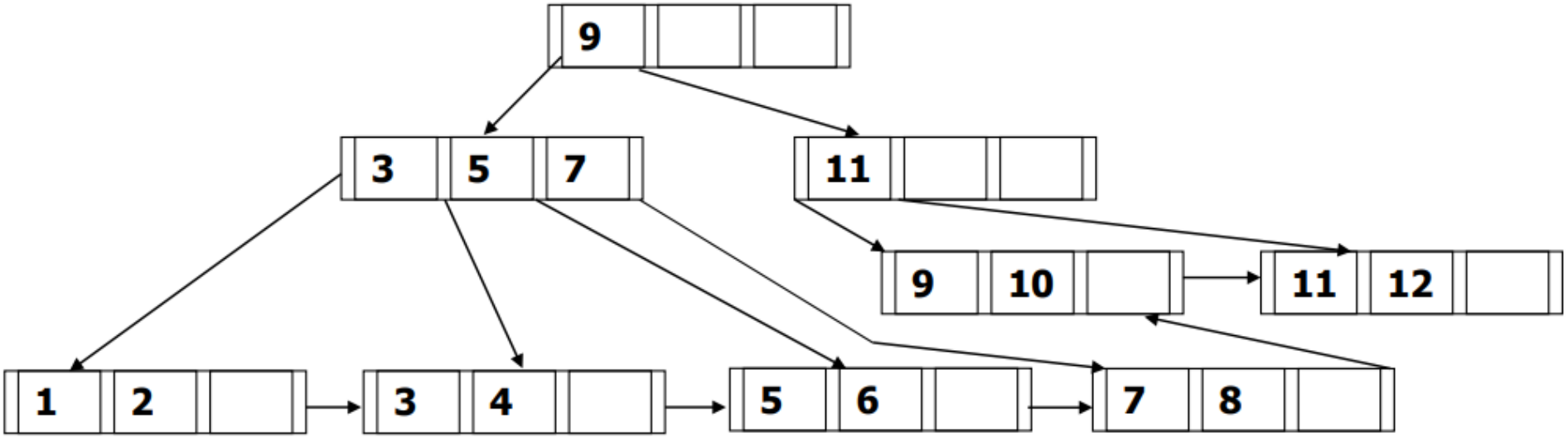
Insert 1, 3, 5, 7, 9, 2, 4, 6, 8, 10



- Insert 10



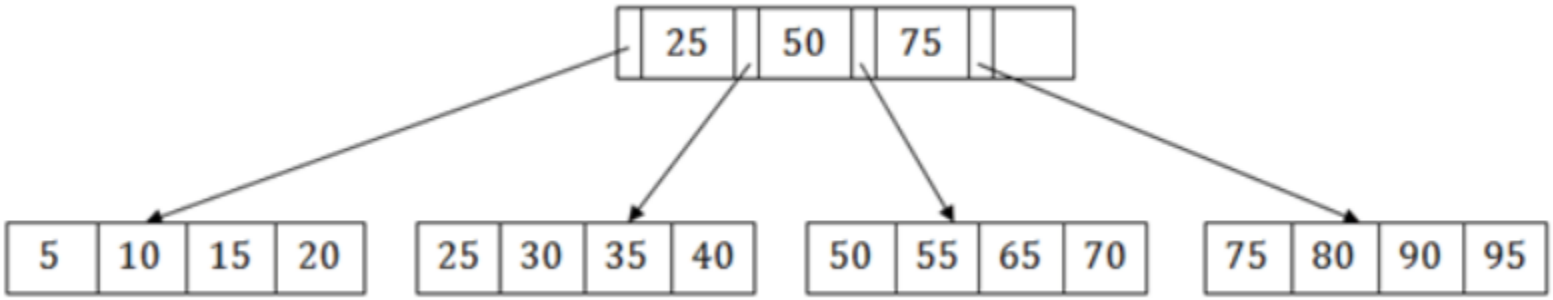
Show the tree after deletions



- Remove 9, 7, 8

Searching a record in B+ Tree

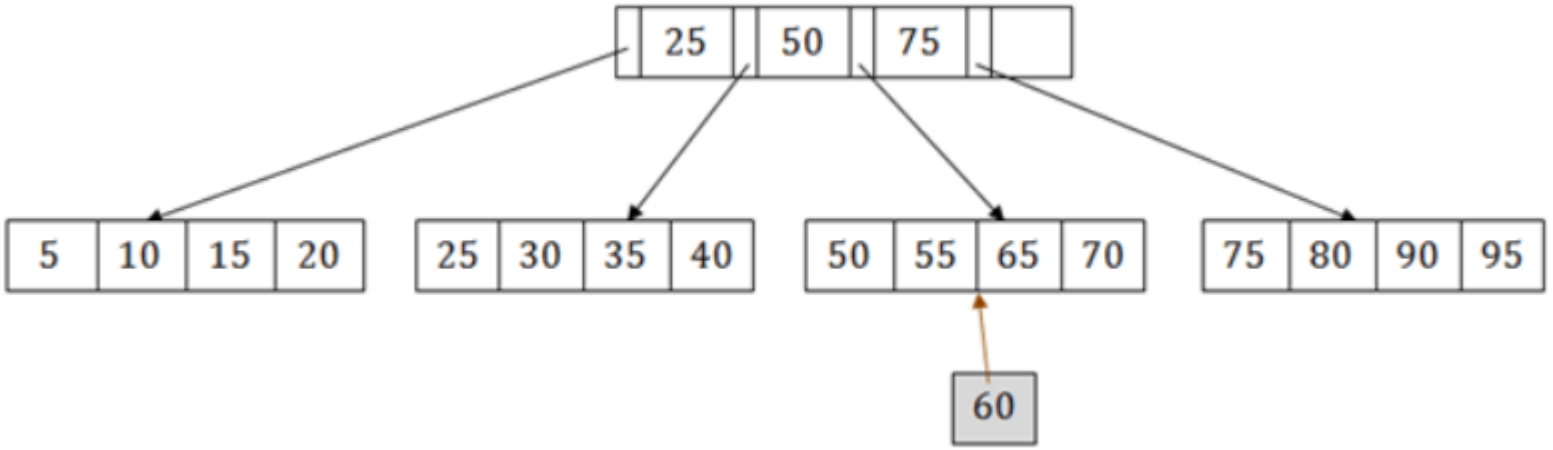
- Suppose we have to search 55 in the below B+ tree structure.



- First, we will fetch for the intermediary node which will direct to the leaf node that can contain a record for 55.
- So, in the intermediary node, we will find a branch between 50 and 75 nodes. Then at the end, we will be redirected to the third leaf node.
- Here DBMS will perform a sequential search to find 55.

B+ Tree Insertion

- Suppose we want to insert a record 60 in the below structure.

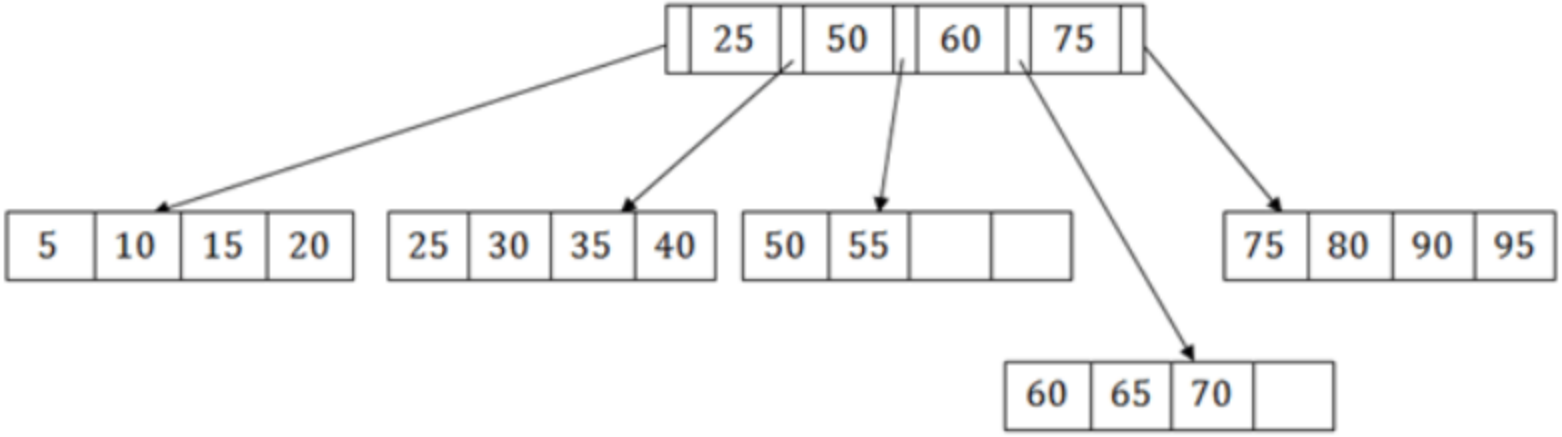


- It will go to the 3rd leaf node after 55. It is a balanced tree and a leaf node of this tree is already full, so we cannot insert 60 there.
- In this case, we have to split the leaf node, so that it can be inserted into tree without affecting the fill factor, balance and order.

B+ Tree Insertion

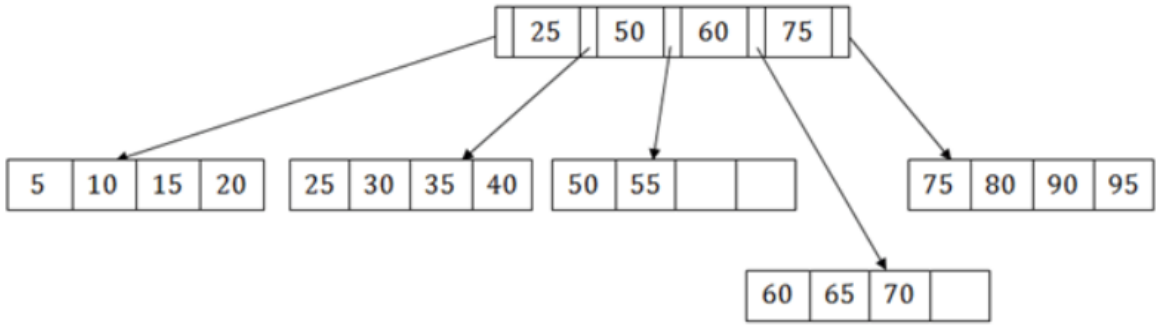
- The 3rd leaf node has the values (50, 55, 60, 65, 70) and its current root node is 50. We will split the leaf node of the tree in the middle so that its balance is not altered. So we can group (50, 55) and (60, 65, 70) into 2 leaf nodes.

- If these two has to be leaf nodes, the intermediate node cannot branch from 50. It should have 60 added to it, and then we can have pointers to a new leaf node.

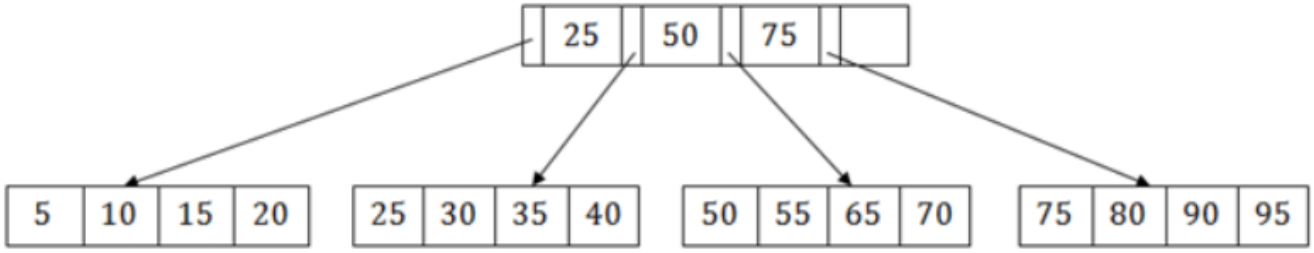


B+ Tree Deletion

- Suppose we want to delete 60 from the above example.



- In this case, we have to remove 60 from the intermediate node as well as from the 4th leaf node too. If we remove it from the intermediate node, then the tree will not satisfy the rule of the B+ tree. So we need to modify it to have a balanced tree.
- After deleting node 60 from above B+ tree and re-arranging the nodes, it will show as follows:

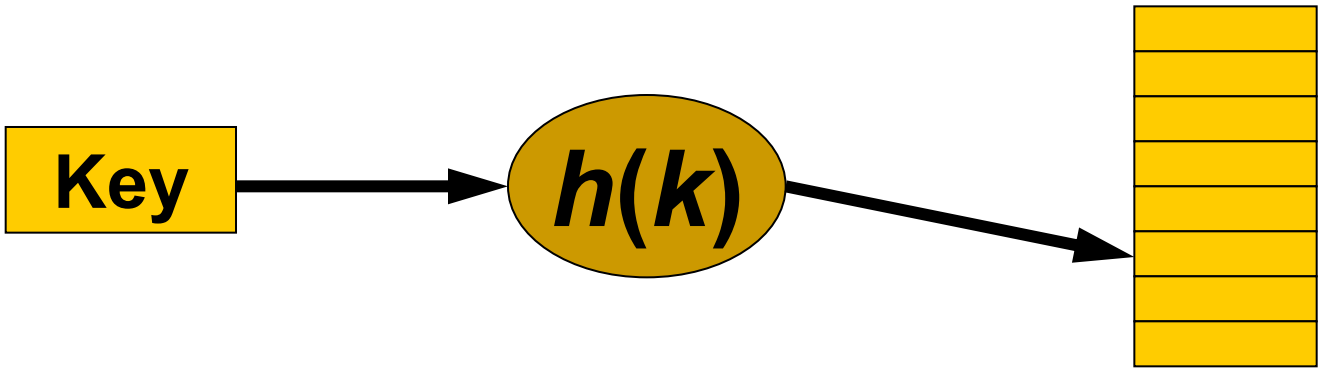


Differences between B-Tree and B+ Tree

B-Tree	B+ Tree
All internal nodes and leaf nodes contain data pointers along with keys.	Only leaf nodes contain data pointers along with keys, internal nodes contain keys only.
There are no duplicate keys.	Duplicate keys are present in this, all internal nodes are also present at leaves.
Leaf nodes are not linked to each other.	Leaf nodes are linked to each other.
Sequential access of nodes is not possible.	All nodes are present at leaves, so sequential access is possible just like a linked list.
Searching for a key is slower.	Searching is faster.
For a particular number of entries, the height of the B-tree is larger.	The height of the B+ tree is lesser than B-tree for the same number of entries.

Hashing

- In database management system, When we want to retrieve a particular data, It becomes very inefficient to search all the index values and reach the desired data. In this situation, Hashing technique comes into picture.
- Hashing is an efficient technique to directly search the location of desired data on the disk without using index structure.
- Data is stored at the data blocks whose address is generated by using hash function.
- The memory location where these records are stored is called as data block or data bucket.



Hash File Organization

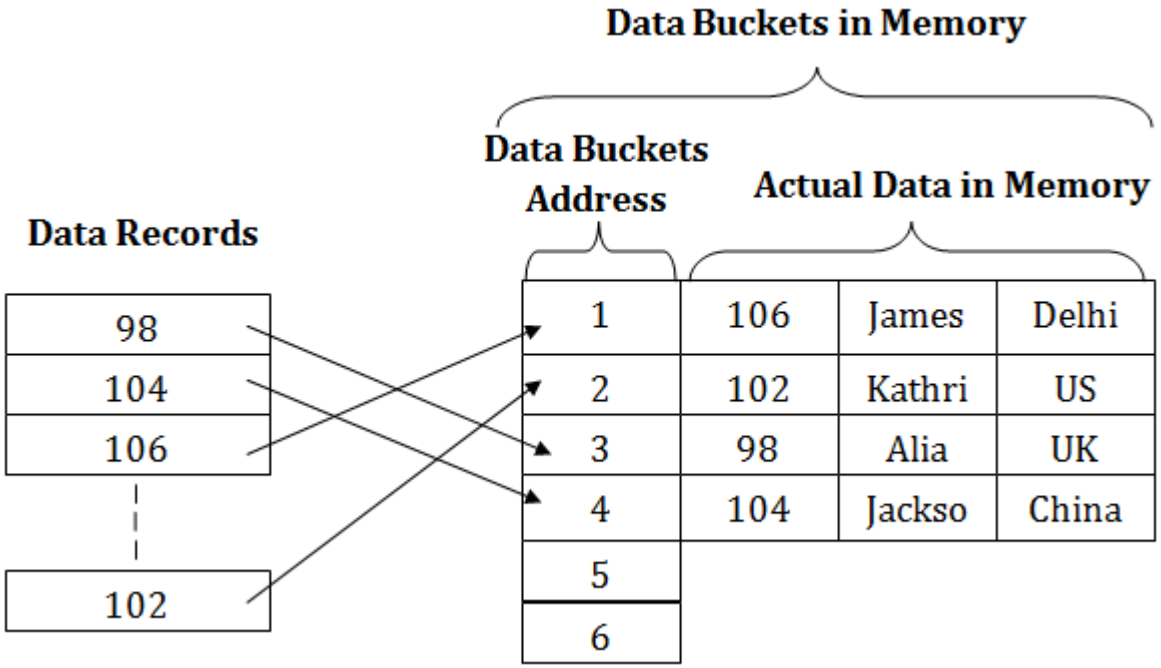
- **Data bucket –** Data buckets are the memory locations where the records are stored.

- **Hash Function –** Hash function is a mapping function that maps all the set of search keys to actual record address.
- Generally, hash function uses primary key to generate the hash index – address of the data block. Hash function can be simple mathematical function to any complex mathematical function.

- **Hash Index-** The prefix of an entire hash value is taken as a hash index.

Static Hashing

- In static hashing, the resultant data bucket address will always be the same. That means if we generate an address for EMP_ID =103 using the hash function mod (5) then it will always result in same bucket address 3. Here, there will be no change in the bucket address.
- Hence in this static hashing, the number of data buckets in memory remains constant throughout.
- In this example, we will have five data buckets in the memory used to store the data.



Operations of Static Hashing

Insert a Record- When a new record is inserted into the table, then we will generate an address for a new record based on the hash key and record is stored in that location.

Delete a Record- To delete a record, we will first fetch the record which is supposed to be deleted. Then we will delete the records for that address in memory.

Update a Record- To update a record, we will first search it using a hash function, and then the data record is updated.

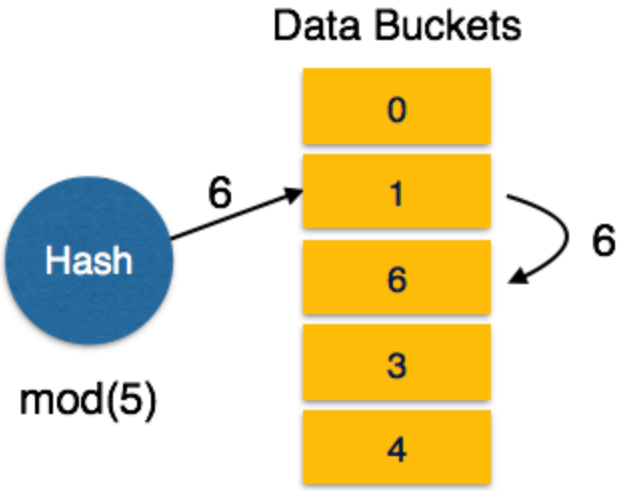
- *If we want to insert some new record into the file but the address of a data bucket generated by the hash function is not empty, or data already exists in that address. This situation in the static hashing is known as bucket overflow. This is a critical situation in this method.*

To overcome this situation, there are various methods. Some commonly used methods are as follows:

1. **Open Hashing or Linear Probing**
2. **Close Hashing or Overflow chaining**

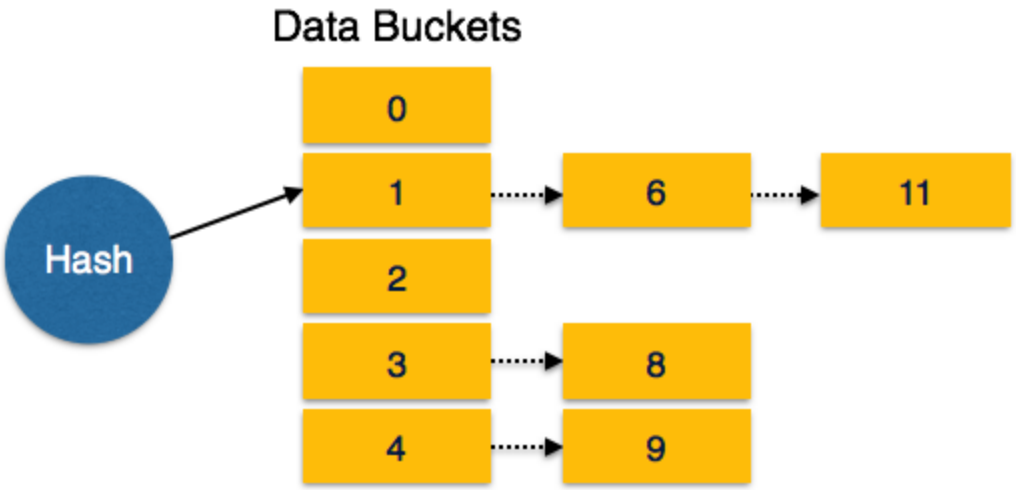
Open Hashing or Linear Probing

When a hash function generates an address at which data is already stored, the next free bucket is allocated to it. This mechanism is called Open Hashing.



Closed Hashing or Overflow Chaining

When buckets are full, a new bucket is allocated for the same hash result and is linked after the previous one. This mechanism is called Closed Hashing.



Dynamic Hashing

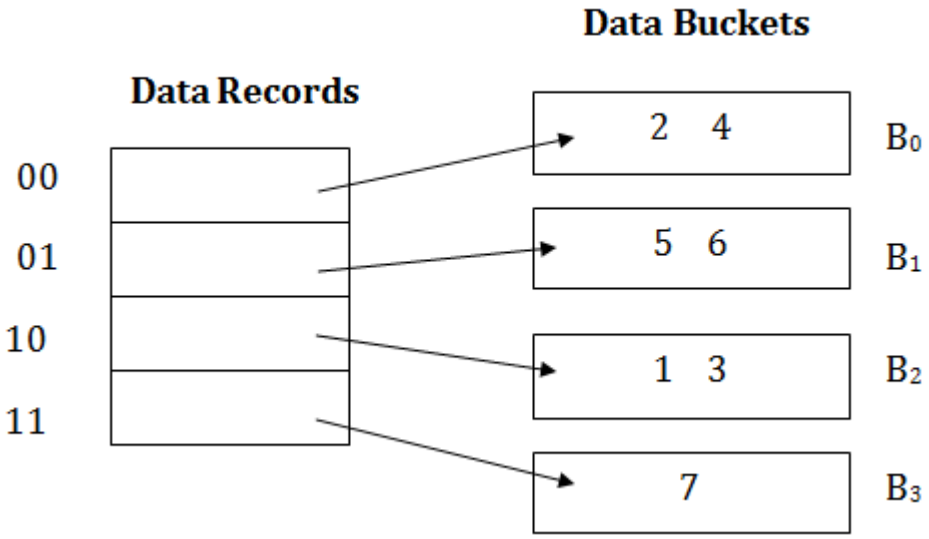
- The dynamic hashing method is used to overcome the problems of static hashing like bucket overflow.
- In this method, data buckets grow or shrink as the records increases or decreases. This method is also known as Extendable hashing method.
- This method makes hashing dynamic, i.e., it allows insertion or deletion without resulting in poor performance.

Dynamic Hashing

- ***Consider the following grouping of keys into buckets, depending on the prefix of their hash address:***

Key	Hash address
1	11010
2	00000
3	11110
4	00000
5	01001
6	10101
7	10111

The last two bits of 2 and 4 are 00. So it will go into bucket B0. The last two bits of 5 and 6 are 01, so it will go into bucket B1. The last two bits of 1 and 3 are 10, so it will go into bucket B2. The last two bits of 7 are 11, so it will go into B3.

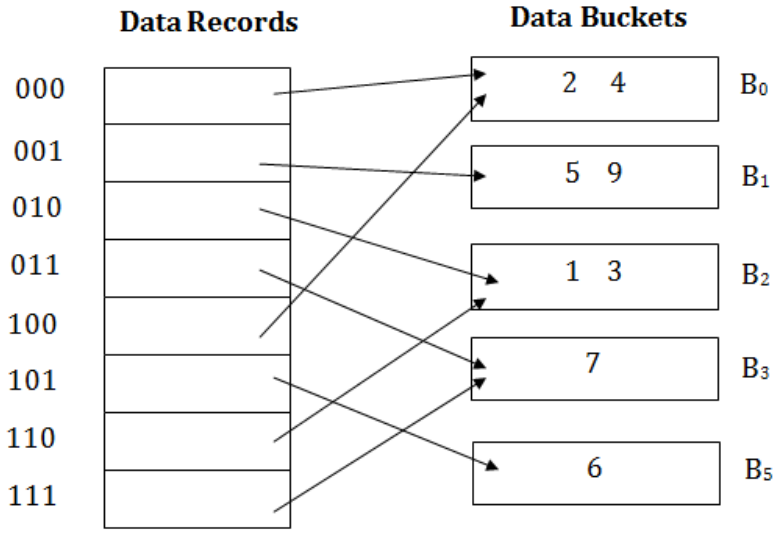


Dynamic Hashing

Insert key 9 with hash address 10001 into the above structure:

- Since key 9 has hash address 10001, it must go into the first bucket. But bucket B1 is full, so it will get split.
- The splitting will separate 5, 9 from 6 since last three bits of 5, 9 are 001, so it will go into bucket B1, and the last three bits of 6 are 101, so it will go into bucket B5.
- Keys 2 and 4 are still in B0. The record in B0 pointed by the 000 and 100 entry because last two bits of both the entry are 00.
- Keys 1 and 3 are still in B2. The record in B2 pointed by the 010 and 110 entry because last two bits of both the entry are 10.
- Key 7 are still in B3. The record in B3 pointed by the 111 and 011 entry because last two bits of both the entry are 11.

Key	Hash address
1	11010
2	00000
3	11110
4	00000
5	01001
6	10101
7	10111



Dynamic Hashing

Advantages of dynamic hashing

- In this method, the performance does not decrease as the data grows in the system. It simply increases the size of memory to accommodate the data.
- In this method, memory is well utilized as it grows and shrinks with the data. There will not be any unused memory lying.
- This method is good for the dynamic database where data grows and shrinks frequently.

Disadvantages of dynamic hashing

- In this method, if the data size increases then the bucket size is also increased. These addresses of data will be maintained in the bucket address table. This is because the data address will keep changing as buckets grow and shrink. If there is a huge increase in data, maintaining the bucket address table becomes tedious.
- In this case, the bucket overflow situation will also occur. But it might take little time to reach this situation than static hashing.

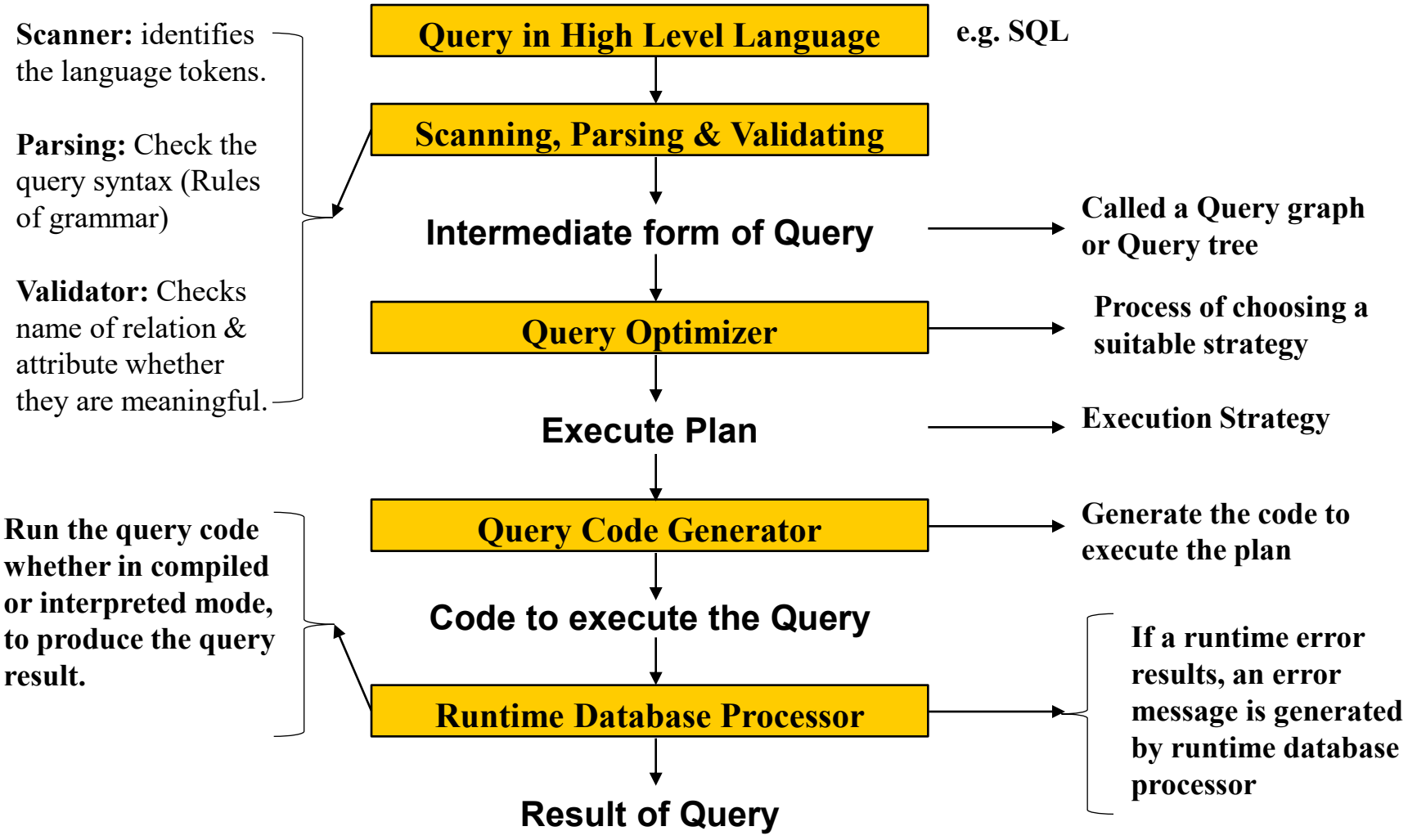
Query Processing:

- Is the process of selecting the most efficient query evaluation plan from among the many strategies usually possible for processing the given query, specially when the query is complex.
- Objective is to make a such system that can minimize the cost of query evaluation by constructing a query evaluation plan.
- One aspect is relational algebra, where system can find the equivalent expression for given relation algebra.
- The goal of query optimization is to reduce the system resources required to fulfill a query and ultimately provide the user with the correct result set faster.

There are broadly two ways a query can be optimized:

- Analyze and transform equivalent relational expressions: Try to minimize the tuple and column counts.
- Using different algorithms for each operation: These underlying algorithms determine how tuples are accessed from the data structures they are stored in.

Steps for Query Processing-



Steps for Query Processing-

- **Query Processing-** Includes translation of high level queries into low level expressions that can be used at physical level of the file system.

- **Evaluation Plans-** The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.

- A relational algebra expression may have many equivalent expressions
  - E.g., $\sigma_{salary<75000}(\prod_{salary}(instructor))$ is equivalent to $\prod_{salary}(\sigma_{salary<75000}(instructor))$

- Each relational algebra operation can be evaluated using one of several different algorithms
  - Correspondingly, a relational-algebra expression can be evaluated in many ways.

  - **Query Optimization-** It is the process in which multiple query evaluation plans are examined and a most efficient plan is identified for execution.
  - e.g. number of tuples in each relation, size of tuples, etc.

Query Optimization:

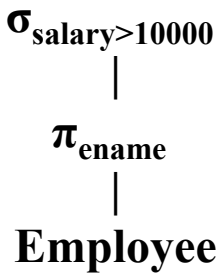
- **Query Tree-** Used to represent relational algebra expression
  - Tree data structure
  - Input relations are represented at leaf node
  - Relational algebra expression are represented at internal node

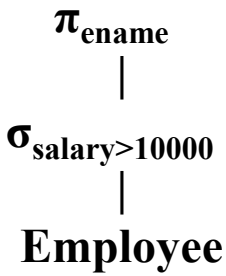
- For example, a user wants to fetch the records of the employees whose salary is greater than 10000. For doing this, the following query is undertaken:

SQL> select ename from Employee where salary>10000;

- It can be translated in the form of relational algebra as-

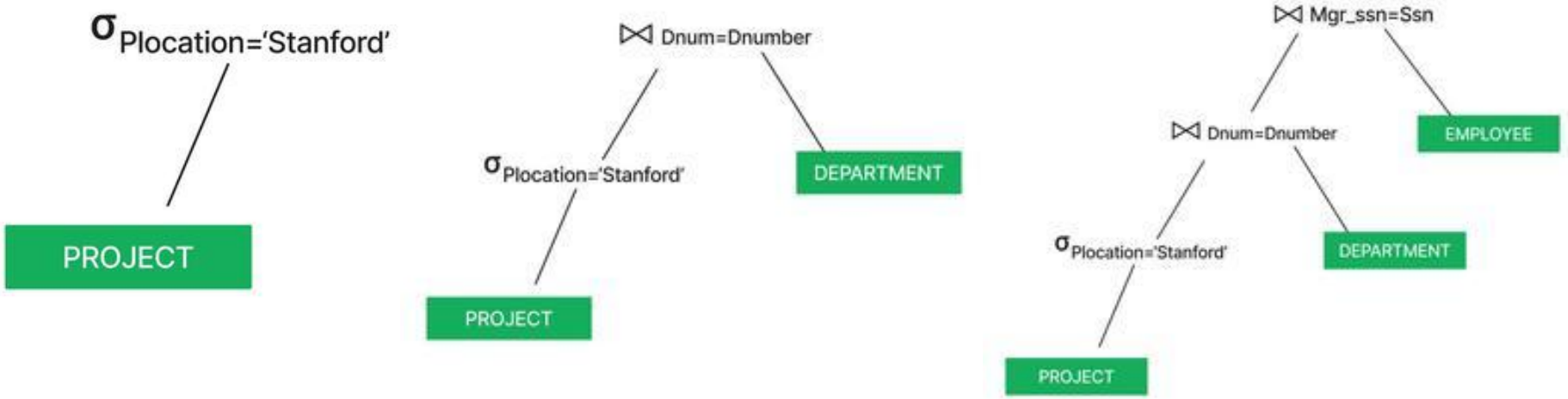
$\sigma_{salary>10000}(\pi_{ename}(Employee))$ **or** $\pi_{ename}(\sigma_{salary>10000}(Employee))$





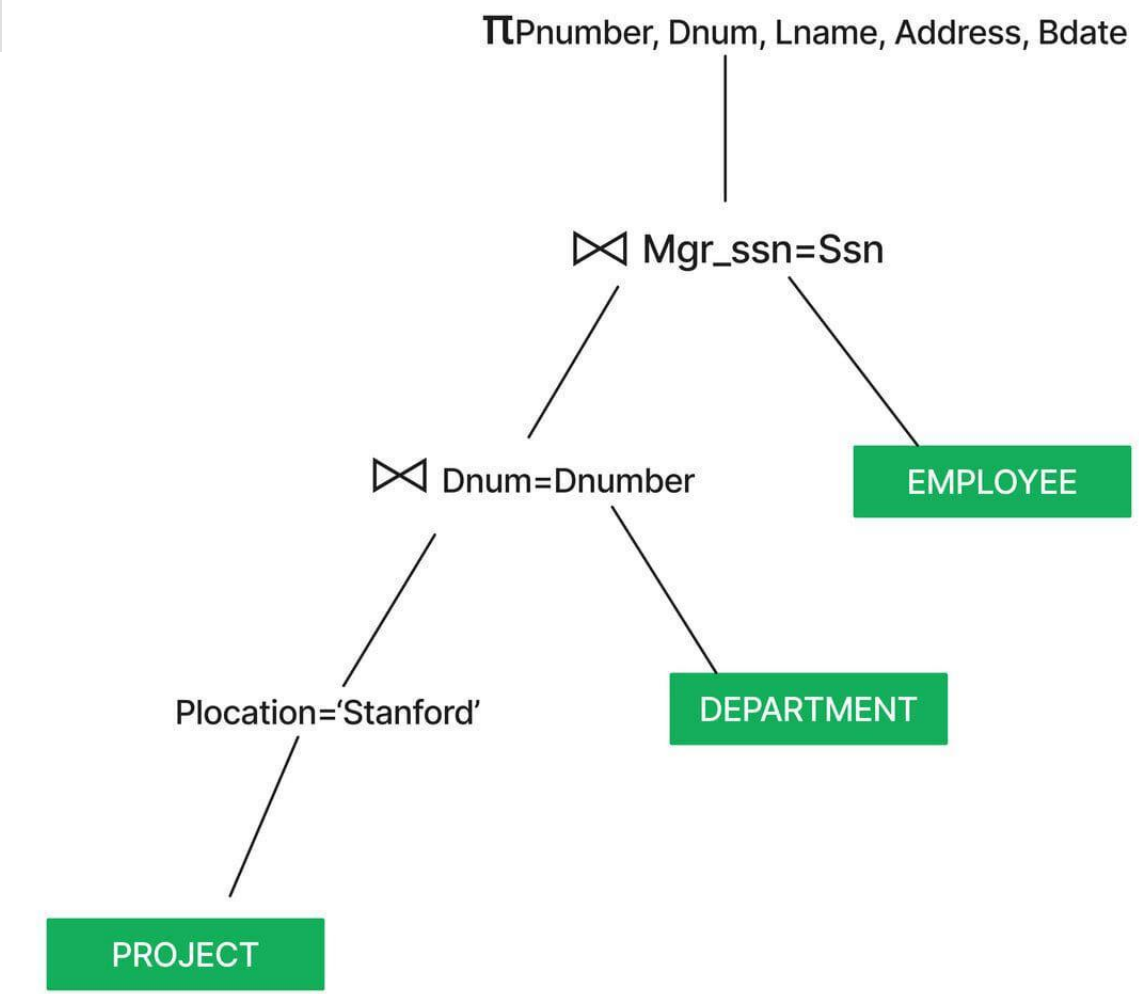
Query Optimization:

$$\pi_{Pnumber,\ Dnum,\ Lname,\ Address,\ Bdate}(((\sigma_{Plocation\ =\ 'Stanford'}(PROJECT)) \bowtie_{Dnum=Dnumber}(DEPARTMENT)) \bowtie_{Mgr_ssn=Ssn}(EMPLOYEE))$$



Query Optimization:

$$\pi_{Pnumber,\ Dnum,\ Lname,\ Address,\ Bdate}(((\sigma_{Plocation\ =\ 'Stanford'}(PROJECT)) \bowtie_{Dnum=Dnumber}(DEPARTMENT)) \bowtie_{Mgr_ssn=Ssn}(EMPLOYEE))$$



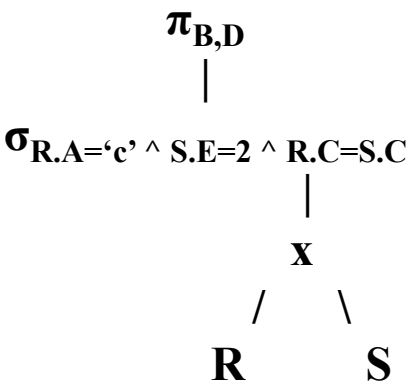
Query Optimization:

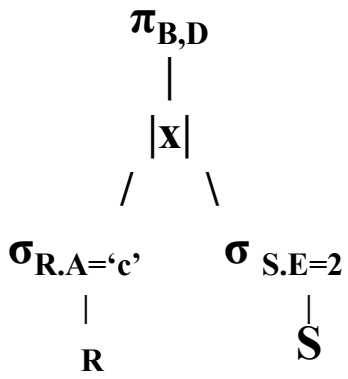
SQL> select B,D from R,S where R.A='c' and S.E=2 and R.C=S.C;

- It can be translated in the form of relational algebra as-

$\pi_{B,D}(\sigma_{R.A='c' \wedge S.E=2 \wedge R.C=S.C}(R \times S))$

$\pi_{B,D}(\sigma_{R.A='c'}(R) \ |x| \ \sigma_{S.E=2}(S))$





Query Optimization:

Examples: Assume the following tables:

instructor(ID, name, dept_name, salary)
teaches(ID, course_id, sec_id, semester, year)
course(course_id, title, dept_name, credits)

Query 1: Find the names of all instructors in the Music department, along with the titles of the courses that they teach

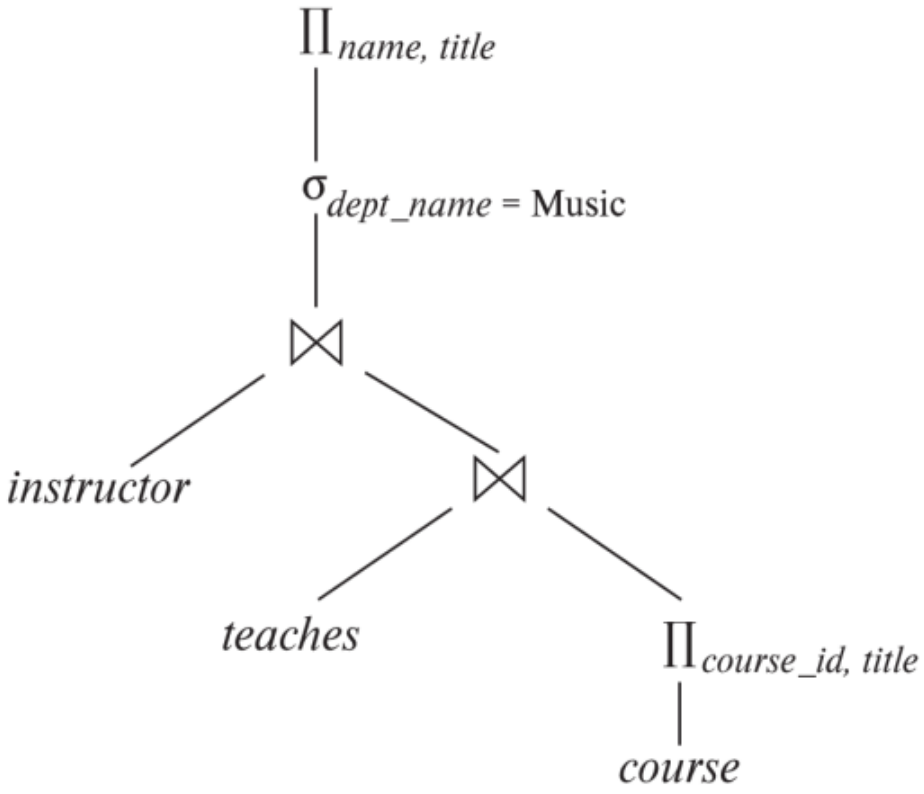
$$\pi_{name,\ title}\left(\sigma_{dept_name="Music"}\left(instructor \bowtie \left(teaches \bowtie \pi_{course_id,\ title}(course)\right)\right)\right)$$

Here, dept_name is a field of only the instructor table. Hence, we can select out the Music instructors before joining the tables, hence reducing query time.

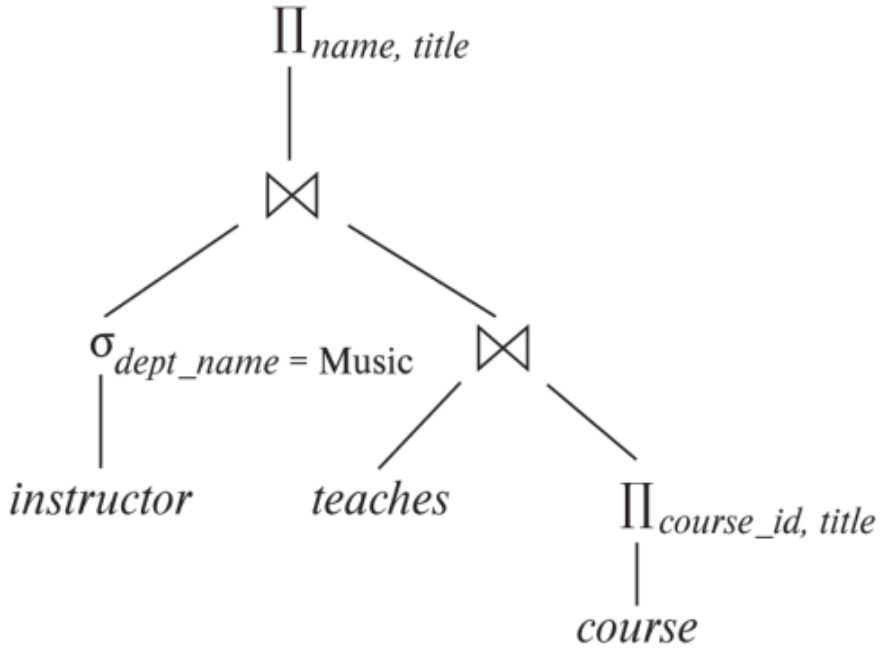
Optimized Query: Performing the selection as early as possible reduces the size of the relation to be joined.

$$\pi_{name,\ title}\left(\left(\sigma_{dept_name="Music"}(instructor)\right) \bowtie \left(teaches \bowtie \pi_{course_id,\ title}(course)\right)\right)$$

Query Optimization:



(a) Initial expression tree



(b) Transformed expression tree

Query Optimization:

Consider the following SQL query:

Code

```
SELECT P.HName, P.Age
FROM Hills AS H, Runners AS P, Runs AS R
WHERE P.HId = R.HId AND R.MId = H.MId AND H.Height > 1200;
```

1. Initial Relational Algebra Expression:

The initial, unoptimized relational algebra expression for this query might look like a Cartesian product followed by selections and a projection:

π HName, Age (σ P.HId = R.HId $\wedge$ R.MId = H.MId $\wedge$ H.Height > 1200 (Hills $\times$ Runners $\times$ Runs))

Query Optimization:

2. Optimization through Predicate Pushdown:

A key optimization technique is predicate pushdown, where selection conditions are moved as early as possible in the query plan. In this case, the condition `H.Height > 1200` only involves the `Hills` relation. Pushing this selection down before the joins can significantly reduce the size of the intermediate relations.

Optimized Expression:

Code

```
π HName, Age ( Runners ⋈ ( Runs ⋈ ( σ Height > 1200 (Hills) ) ) )
```

Here, `⋈` represents the natural join operator. The selection `σ Height > 1200 (Hills)` is performed first, filtering `Hills` before joining it with `Runs` and then `Runners`. This reduces the number of tuples involved in the subsequent join operations, leading to a more efficient execution.

Thank You